

ITNS 414 — Cyber Forensics

Term Project — Final Report

Ransomware Incident Investigation

Using Memory Forensics

A Case Study of the OtterCTF 2018 Dataset

Authors

Ahmed Mohamed Ebaid (*Team Leader*)

Omar Ahmed Hamed

Shady Mohamed Afifi

Submission Date: April 27, 2026

Project Deadline: May 17, 2026

All analysis was conducted inside isolated virtual machines. No active malware was executed on host systems.

Executive Summary

This report documents a four-phase memory forensics investigation conducted against the OtterCTF 2018 dataset — a publicly available Windows 7 SP1 x64 memory image (`OtterCTF.vmem`, 2 GB) containing a complete ransomware infection. The investigation was performed inside an isolated Kali Linux virtual machine using the Volatility 2.6.1 and Volatility 3 frameworks, supported by Hashcat, YARA, SQLite, and standard Linux forensic utilities.

The investigation reconstructed the entire attack lifecycle from initial reconnaissance through file encryption and ransom note display. The infected user (`Rick`) downloaded a pirated torrent file disguised as “Rick And Morty season 1 download.exe” on July 7, 2018. On August 4, 2018, this dropper self-extracted a .NET ransomware executable (`vmware-tray.exe`) into the user’s temp directory. The ransomware injected shellcode and a hidden DLL into its own process, generated an AES encryption key in memory, encrypted user files with a .WINDOWS extension, and displayed a ransom note — all in under two minutes.

Memory forensics yielded the following critical recoveries:

- Full process injection evidence (4 RWX memory regions and reflective DLL loading)
- AES encryption key material recovered from process memory
- NTLM credential hash extracted from LSASS and successfully cracked
- Complete reconstruction of the user-driven infection path via Chrome browser history
- Command-and-control (C2) network connection to `77.102.199.102:7575`

The investigation findings were mapped to nine MITRE ATT&CK techniques across seven tactical categories, validated against the Critical Infrastructure Ransomware Attacks (CIRA) dataset, and translated into five evidence-based prevention measures.

Investigation outcome

Status: All investigation objectives met. The complete attack kill chain was reconstructed, malware artifacts were extracted and analysed, victim credentials were recovered, and prevention recommendations were formulated. Total elapsed time from dropper execution to ransom note display: approximately 75 seconds.

Contents

Executive Summary	i
1 Introduction	1
1.1 Background	1
1.2 Research Question and Objective	1
1.3 Scope and Constraints	1
1.3.1 In scope	2
1.3.2 Out of scope	2
1.4 Team Structure and Task Delegation	2
1.5 Report Structure	2
2 Methodology	4
2.1 Investigation Framework	4
2.2 Tooling	4
2.3 Evidence Handling and Chain of Custody	5
2.4 Case Folder Structure	5
2.5 Reproducibility	6
2.6 Ethical Considerations	6
3 Phase 0 — Environment Setup	7
3.1 Overview	7
3.2 Step 0.1 — Case Folder Structure	7
3.3 Step 0.2 — Isolated Virtual Machine	8
3.4 Step 0.3 — Tool Installation and Version Verification	10
3.5 Step 0.4 — Evidence Hashing	11
3.6 Step 0.5 — Working Copy Creation and Read-Only Protection	12
3.7 Phase 0 Summary	13
4 Phase 1 — System Profiling	14
4.1 Overview	14
4.2 Step 1.1 — Image Identification	14
4.3 Step 1.2 — Hostname and User Context	15
4.4 Step 1.3 — Process Enumeration	17
4.5 Step 1.4 — Network Connections	19

4.6	Phase 1 Summary	20
5	Phase 2 — Attack Vector Identification	21
5.1	Overview	21
5.2	Step 2.1 — Suspicious Process Command Line	21
5.3	Step 2.2 — Chrome History File Discovery	22
5.4	Step 2.3 — Chrome History Extraction	23
5.5	Step 2.4 — Browser History SQL Query	24
5.6	Step 2.5 — Dropper File Recovery	25
5.7	Step 2.6 — Dropper Hashing	26
5.8	Step 2.7 — VirusTotal Hash Lookup	26
5.9	Phase 2 Summary	27
6	Phase 3 — Malware Analysis	29
6.1	Overview	29
6.2	Step 3.1 — Code Injection Detection (malfind)	29
6.3	Step 3.2 — DLL Analysis and Hidden Library Detection	32
6.4	Step 3.3 — Process Memory Dump	33
6.5	Step 3.4 — String Analysis and IOC Extraction	34
6.5.1	Pass 1 — Cryptocurrency wallet search	34
6.5.2	Pass 2 — Encryption library and password search	35
6.5.3	Pass 3 — Ransom note recovery (UTF-16)	36
6.5.4	Pass 4 — Encryption key candidate (UTF-16)	37
6.6	Step 3.5 — Credential Recovery (hashdump and hashcat)	38
6.6.1	First crack attempt — rockyou.txt	38
6.6.2	Second crack attempt — targeted wordlist	39
6.7	Step 3.6 — UserAssist Registry Analysis	40
6.8	Step 3.7 — Persistence Check (Run Keys)	41
6.9	Phase 3 Summary	42
7	Phase 4 — Kill Chain Reconstruction and Prevention	44
7.1	Overview	44
7.2	Step 4.1 — Master Attack Timeline	44
7.3	Step 4.2 — MITRE ATT&CK Mapping	46
7.4	Step 4.3 — CIRA Cross-Reference	47
7.5	Step 4.4 — Evidence-Based Prevention Measures	48
7.5.1	PM-1: Application allowlisting (AppLocker / WDAC)	48
7.5.2	PM-2: Egress filtering of P2P/torrent traffic	48
7.5.3	PM-3: Credential Guard and LSA Protection	48
7.5.4	PM-4: EDR with behaviour-based injection detection	48
7.5.5	PM-5: Security awareness training and email/web filtering	49
7.6	Phase 4 Summary	49
8	Conclusions	50

8.1	Investigation Outcome	50
8.2	Key Lessons Learned	50
8.2.1	Memory holds what disk forgets	50
8.2.2	User behaviour is the perimeter	51
8.2.3	The 75-second problem	51
8.2.4	Negative findings are findings	51
8.3	Limitations	51
8.4	Future Work	51
8.5	Closing Statement	52
9	Appendices	53
9.1	Appendix A — Complete Command Reference	53
9.1.1	Phase 0 — Environment setup	53
9.1.2	Phase 1 — System profiling	53
9.1.3	Phase 2 — Attack vector identification	54
9.1.4	Phase 3 — Malware analysis	55
9.2	Appendix B — Output File Inventory	56
9.3	Appendix C — Indicators of Compromise	57
9.4	Appendix D — Glossary	58
9.5	Appendix E — Acknowledgements	59

List of Figures

3.1	Case folder structure	8
3.2	Kali VM hardware configuration	9
3.3	Clean baseline snapshot	10
3.4	Forensic tool versions	11
3.5	Original evidence hashes	12
3.6	Working copy hash verification	12
3.7	Read-only protection on original evidence	13
4.1	Volatility 2 imageinfo output	15
4.2	Volatility 3 windows.info output	15
4.3	Hostname extraction	16
4.4	User context	17
4.5	Process tree	18
4.6	Filtered process list	18
4.7	Network connections	19
5.1	Suspicious command lines	22
5.2	Chrome history file objects	23
5.3	Chrome history dump	23
5.4	Torrent search history	24
5.5	MEGA dropper download URL	25
5.6	Dropper file location	25
5.7	Dropper hashes and signature	26
5.8	VirusTotal hash lookup	27
6.1	Malfind RWX region at 0x670000	30
6.2	Malfind RWX region at 0x510000	31
6.3	DLL list of vmware-tray.exe	32
6.4	Hidden DLL detected	33
6.5	Process dump confirmation	34
6.6	Cryptocurrency search results	35
6.7	AES encryption strings recovered	36
6.8	Ransom note text recovered from memory	37
6.9	Encryption key candidate recovered	37

6.10 NTLM hash extraction	38
6.11 Rockyou exhausted, no crack	39
6.12 Targeted attack succeeded	40
6.13 UserAssist execution evidence	41
6.14 Run key enumeration	42
7.1 Reconstructed attack timeline	45
7.2 MITRE ATT&CK Navigator coverage	46

List of Tables

1.1	Team roles and responsibilities	2
2.1	Four-phase investigation methodology	4
2.2	Forensic tools used in the investigation	5
3.1	Analysis VM specifications	8
3.2	Phase 0 completion summary	13
4.1	Phase 1 completion summary	20
5.1	Phase 2 completion summary	28
6.1	UserAssist execution timeline	41
6.2	Phase 3 completion summary	43
7.1	MITRE ATT&CK techniques observed in the OtterCTF investigation	47
7.2	CIRA dataset cross-reference for observed TTPs	47
7.3	Evidence-based prevention measures summary	49
7.4	Phase 4 completion summary	49
9.1	Investigation output files	57
9.2	Indicators of compromise	57

Chapter 1

Introduction

1.1 Background

Ransomware remains one of the most disruptive cyber threats of the past decade, causing billions of dollars in damages annually across critical infrastructure, healthcare, education, and private enterprise sectors. Unlike traditional disk-based forensic analysis, modern ransomware investigations require deep memory analysis, because crucial artifacts — including injected DLLs, decrypted strings, encryption keys, and stolen credentials — exist only transiently in volatile memory and are destroyed at system shutdown.

The OtterCTF 2018 dataset is a publicly available, self-contained capture of a fully compromised Windows 7 SP1 x64 system. It was originally released as part of the OtterCTF capture-the-flag competition and has since become a standard educational dataset for memory forensics training. The dataset comprises a single 2 GB raw memory image (`OtterCTF.vmem`) captured shortly after a ransomware infection completed its encryption routine.

This investigation uses the OtterCTF dataset to demonstrate a complete, structured memory forensics workflow that bridges the gap between theoretical knowledge and practical incident response.

1.2 Research Question and Objective

Core research question: How can memory forensics be utilised to reconstruct the full lifecycle of a ransomware attack — from the initial infection vector through to file encryption — and what actionable enterprise prevention measures emerge from this analysis?

Investigation objective: To perform a structured, four-phase memory forensics investigation of a ransomware-compromised Windows 7 system, identifying all attack stages and producing evidence-based prevention recommendations.

1.3 Scope and Constraints

1.3.1 In scope

- Volatile memory analysis of `OtterCTF.vmem` using Volatility 2 and Volatility 3
- Identification of the initial infection vector through browser history, command line, and file scans
- Extraction and analysis of the malicious processes, including process injection detection
- Recovery of credentials, encryption keys, and ransom note text from process memory
- Mapping of observed techniques to the MITRE ATT&CK framework (Enterprise matrix, version 14)
- Cross-reference with the CIRA (Critical Infrastructure Ransomware Attacks) dataset
- Formulation of evidence-based prevention measures

1.3.2 Out of scope

- Network packet capture analysis (no PCAP file is provided with the OtterCTF dataset)
- Disk image forensic analysis (no disk image is provided)
- Reverse engineering of the malware binary at the assembly level
- Decryption of the encrypted user files (the original key derivation routine cannot be reconstructed without the binary)

1.4 Team Structure and Task Delegation

The investigation was conducted by a three-member team with cross-trained roles to ensure all members understood the complete forensic process. Tasks were divided as follows:

Table 1.1. Team roles and responsibilities

Member	Role and primary tasks
Ahmed M. Ebaid (<i>Team Leader</i>)	Project integration and prevention strategist. Manages timelines, compiles the final report, maps findings to MITRE ATT&CK, and formulates data-driven prevention measures with reference to the CIRA dataset.
Omar A. Hamed	Initial access and vector analyst. Analyses browser history, network connections, and command-line execution to trace the initial malicious dropper.
Shady M. Afifi	Artifact and malware analyst. Extracts loaded DLLs, dumps suspicious processes, analyses injected code, and recovers NTLM credentials.

1.5 Report Structure

This report is organised into nine chapters following a chronological investigation flow:

- **Chapter 1 (Introduction)** establishes scope, objectives, and team structure.
- **Chapter 2 (Methodology)** describes the four-phase investigation framework, tools, and evidence handling procedures.
- **Chapter 3 (Phase 0 — Environment Setup)** documents the isolated analysis environment, tool installation, and chain-of-custody hashing of the original evidence.
- **Chapter 4 (Phase 1 — System Profiling)** captures the baseline system context, host-name, user accounts, processes, and network connections.
- **Chapter 5 (Phase 2 — Attack Vector Identification)** traces the infection from browser history through dropper download to suspicious process execution.
- **Chapter 6 (Phase 3 — Malware Analysis)** performs process injection detection, memory string analysis, encryption key recovery, and credential extraction.
- **Chapter 7 (Phase 4 — Kill Chain Reconstruction and Prevention)** compiles the master attack timeline, maps techniques to MITRE ATT&CK, cross-references the CIRA dataset, and formulates prevention measures.
- **Chapter 8 (Conclusions)** summarises findings and discusses lessons learned.
- **Chapter 9 (Appendices)** contains the complete command reference, output file inventory, and supplementary technical detail.

Chapter 2

Methodology

2.1 Investigation Framework

The investigation followed a structured four-phase methodology designed to progress from broad situational awareness to fine-grained malware analysis and finally to actionable prevention recommendations. Each phase has clearly defined objectives, deliverables, and a screenshot evidence trail to support reproducibility.

Table 2.1. Four-phase investigation methodology

Phase	Name	Primary objective
0	Environment setup	Establish isolated analysis VM, hash and protect original evidence, verify tool versions
1	System profiling	Identify OS, hostname, users, running processes, network connections, and process tree
2	Vector identification	Trace the infection from browser history through file download to dropper execution
3	Malware analysis	Analyse the ransomware process: injection, hidden DLLs, memory strings, encryption key, credential dumping
4	Kill chain and prevention	Build attack timeline, map to MITRE ATT&CK, cross-reference CIRA, derive prevention measures

2.2 Tooling

All analysis was performed inside an isolated Kali Linux 2024 virtual machine (8 GB RAM, 4 vCPU, 70 GB disk, host-only network). The following tools were installed and verified before evidence was loaded.

Table 2.2. Forensic tools used in the investigation

Tool	Version	Primary use
Volatility 2	2.6.1	Primary memory analysis framework (Win7SP1x64 profile)
Volatility 3	2.27.0	Symbol-based memory analysis, secondary verification
YARA	4.5.5	Pattern matching against memory dumps
Hashcat	7.1.2	NTLM hash cracking with rockyou.txt and targeted wordlists
SQLite	3.46.1	Chrome browser history database analysis
GNU strings	2.46	ASCII and Unicode string extraction from process dumps
Binwalk	2.4.3	Embedded file signature analysis
Foremost	1.5.7	File carving (held in reserve)
Sleuthkit	current	Disk image utilities (held in reserve)

2.3 Evidence Handling and Chain of Custody

Best-practice evidence handling principles were applied throughout the investigation:

1. The original `OtterCTF.vmem` file was hashed with both MD5 and SHA-256 immediately after acquisition and before any analysis.
2. A working copy was created, and its hash was independently verified to match the original.
3. The original evidence was set to read-only file permissions to prevent accidental modification.
4. All commands and outputs were logged to timestamped files for reproducibility.
5. All analysis was conducted exclusively on the working copy; the original was never opened by an analysis tool.

Evidence integrity hashes

File: `OtterCTF.vmem` (2,147,483,648 bytes)
MD5: `ad51f4ada4151eab76f2dce8dea69868`
SHA-256: `5b3d8a9f9c96581a821c19b71dd6aa80dd299fc674b818f443f3a6fb5495a872`

2.4 Case Folder Structure

A standardised case folder structure was created at `~/Cases/OtterCTF2018/` to organise all artifacts:

```
~/Cases/OtterCTF2018/
|-- evidence/      Original read-only memory image and hash files
|-- working/       Working copy used for all Volatility commands
|-- output/        All command output files (text, dumps, extracts)
|-- logs/          Tool execution logs
|-- reports/       Final report drafts
|-- screenshots/   Evidence screenshots organised by phase
```

Listing 2.1. Case folder organisation

2.5 Reproducibility

Every command executed during the investigation was logged with a timestamp via the Linux `date` command. The Volatility plugin invocations, file paths, and `grep` filters used for each step are reproduced verbatim in the relevant chapter sections of this report. A complete command reference is provided in Appendix [9.1](#).

2.6 Ethical Considerations

The OtterCTF dataset is publicly released for educational use. The compromised system belonged to a fictional user (**Rick**) and contains no real personally identifiable information. All credentials recovered during this investigation are part of the publicly known CTF challenge and pose no security risk. No live malware was executed at any point; analysis was confined to passive examination of the static memory image.

Chapter 3

Phase 0 — Environment Setup

3.1 Overview

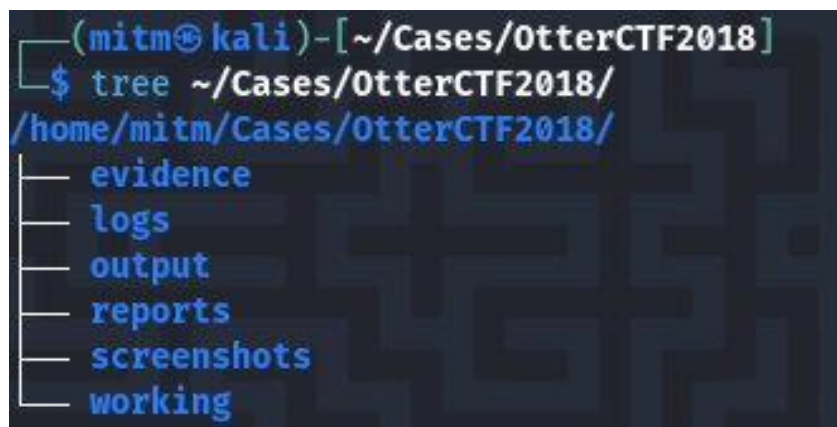
Phase 0 established the foundation for the investigation. The objectives were to create an isolated analysis environment, establish chain-of-custody hashes for the original evidence, build a clean working copy, and verify that all required forensic tools were installed and operational. This phase produced seven evidence screenshots documenting each setup step.

3.2 Step 0.1 — Case Folder Structure

A hierarchical case folder was created to organise all artifacts produced during the investigation. The structure separates original evidence from working copies, output files, screenshots, and reports, in line with industry best practice for forensic case management.

```
mkdir -p ~/Cases/OtterCTF2018/{evidence,logs,output,reports,
    screenshots,working}
tree ~/Cases/OtterCTF2018/
```

Listing 3.1. Command: case folder creation



```

(mitm@kali)-[~/Cases/OtterCTF2018]
$ tree ~/Cases/OtterCTF2018/
/home/mitm/Cases/OtterCTF2018/
├── evidence
├── logs
├── output
├── reports
├── screenshots
└── working

```

Figure 3.1. Case folder hierarchy created at `~/Cases/OtterCTF2018/`, showing the six standard subdirectories.

3.3 Step 0.2 — Isolated Virtual Machine

The analysis environment was a freshly installed Kali Linux virtual machine with the following specifications, configured to be completely isolated from the host network and any production environment.

Table 3.1. Analysis VM specifications

Property	Value
Operating system	Kali Linux 2024
RAM	8 GB
vCPU cores	4
Disk	70 GB SCSI
Network adapter	Host-only (no internet during analysis)
USB controller	Present (for evidence transfer)
Snapshot	Clean baseline taken before evidence load

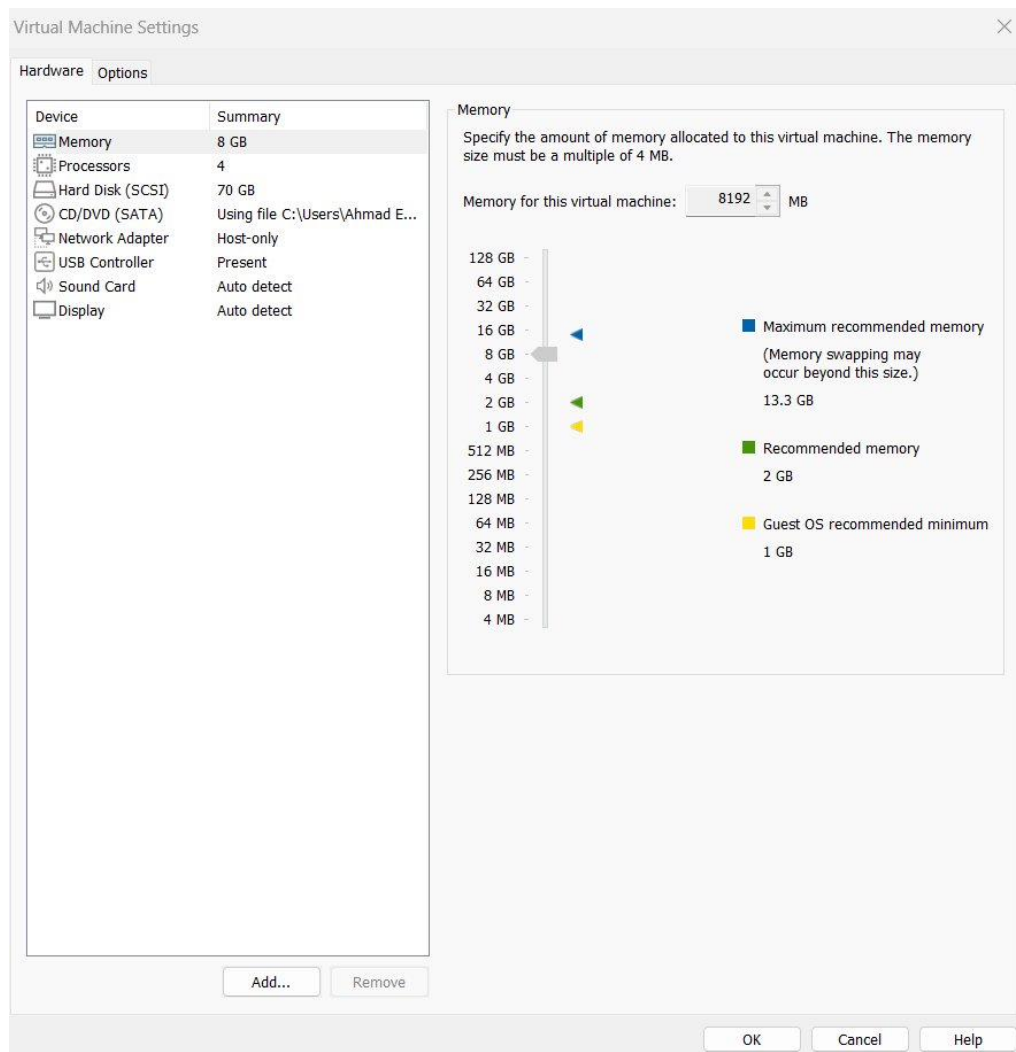


Figure 3.2. VMware virtual machine settings showing 8 GB RAM, 4 vCPU, 70 GB disk, and host-only network adapter.

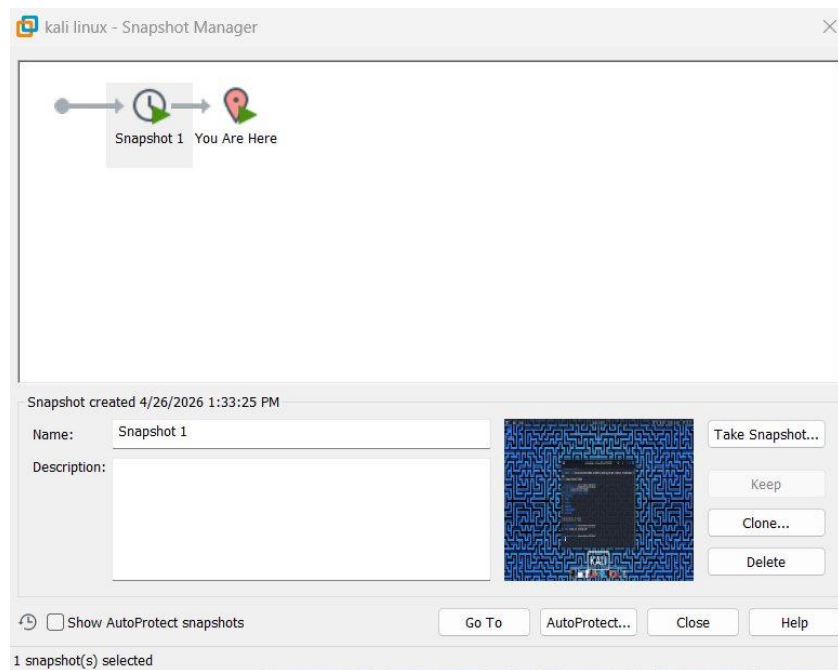


Figure 3.3. VMware Snapshot Manager showing the clean baseline snapshot taken before evidence was loaded, allowing the environment to be reverted to a pristine state at any time.

3.4 Step 0.3 — Tool Installation and Version Verification

All forensic tools required for the investigation were installed and version-verified before evidence was loaded. Verifying tool versions before analysis is essential for reproducibility — different tool versions can produce subtly different outputs.

```

=== Volatility 2 ===
Volatility Foundation Volatility Framework 2.6.1
Win7SP1x64 - A Profile for Windows 7 SP1 x64
=== Volatility 3 ===
usage: vol [-h] [-c CONFIG] [--parallelism [{processes,threads,off}]]
          [-e EXTEND] [-p PLUGIN_DIRS] [-s SYMBOL_DIRS] [-v] [-l LOG]
=== YARA ===
4.5.5
=== Hashcat ===
v7.1.2
=== SQLite ===
3.46.1 2024-08-13 09:16:08 c9c2ab54ba1f5f46360f1b4f35d849cd3f080e6fc2b6c60e91b16c63f69aalt1 (64-bit)
=== Strings ===
GNU strings (GNU Binutils for Debian) 2.46
Copyright (C) 2026 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.
=== Binwalk ===

Binwalk v2.4.3
=== Foremost ===
1.5.7
This program is a work of the US Government. In accordance with 17 USC 105,
copyright protection is not available for any work of the US Government.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
=== Sleuthkit ===
mmls: invalid option -- '-'
Unknown argument
usage: mmls [-i imgtype] [-b dev_sector_size] [-o imgoffset] [-BrV] [-aAmM] [-t vstype] image [images]
  -t vstype: The type of volume system (use '-t list' for list of supported types)
  -i imgtype: The format of the image file (use '-i list' for list supported types)
  -b dev_sector_size: The size (in bytes) of the device sectors
  -o imgoffset: Offset to the start of the volume that contains the partition system (in sectors)
  -B: print the rounded length in bytes
  -r: recurse and look for other partition tables in partitions (DOS Only)
  -v: verbose output
  -V: print the version
Unless any of these are specified, all volume types are shown
  -a: Show allocated volumes
  -A: Show unallocated volumes
  -m: Show metadata volumes
  -M: Hide metadata volumes

```

Figure 3.4. Verified versions of Volatility 2 (2.6.1), Volatility 3 (2.27.0 reported elsewhere), YARA 4.5.5, Hashcat 7.1.2, SQLite 3.46.1, GNU strings 2.46, Binwalk 2.4.3, Foremost 1.5.7, and Sleuthkit. The Win7SP1x64 profile required for Volatility 2 is also confirmed loaded.

3.5 Step 0.4 — Evidence Hashing

Immediately after the original `OtterCTF.vmem` file was placed in the `evidence/` directory, both MD5 and SHA-256 cryptographic hashes were computed. These hashes establish the chain-of-custody baseline against which all later integrity verifications can be performed.

```

cd ~/Cases/OtterCTF2018/evidence
date
md5sum OtterCTF.vmem | tee OtterCTF.vmem.md5
sha256sum OtterCTF.vmem | tee OtterCTF.vmem.sha256

```

Listing 3.2. Command: original evidence hashing

```

(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/evidence]
└─$ date
md5sum OtterCTF.vmem | tee OtterCTF.vmem.md5
sha256sum OtterCTF.vmem | tee OtterCTF.vmem.sha256
Sun Apr 26 03:13:01 PM EEST 2026
ad51f4ada4151eab76f2dce8dea69868 OtterCTF.vmem
5b3d8a9f9c96581a821c19b71dd6aa80dd299fc674b818f443f3a6fb5495a872 OtterCTF.vmem

```

Figure 3.5. MD5 and SHA-256 cryptographic hashes computed against the original `OtterCTF.vmem` file. These values form the chain-of-custody baseline.

Hash baseline

MD5: ad51f4ada4151eab76f2dce8dea69868

SHA-256: 5b3d8a9f9c96581a821c19b71dd6aa80dd299fc674b818f443f3a6fb5495a872

File size: 2,147,483,648 bytes (2.0 GB)

3.6 Step 0.5 — Working Copy Creation and Read-Only Protection

A working copy of the evidence was created in the `working/` directory under the name `OtterCTF_working.vmem`. The SHA-256 hash of the working copy was independently computed and confirmed to match the original. The original evidence was then set to read-only permissions (`chmod 444`) to prevent any accidental modification during the investigation.

```

cp ~/Cases/OtterCTF2018/evidence/OtterCTF.vmem \
  ~/Cases/OtterCTF2018/working/OtterCTF_working.vmem

date
echo "=== ORIGINAL ===" && sha256sum ~/Cases/OtterCTF2018/evidence/
  OtterCTF.vmem
echo "=== WORKING COPY ===" && sha256sum ~/Cases/OtterCTF2018/working
  /OtterCTF_working.vmem

```

Listing 3.3. Command: working copy verification

```

(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/evidence]
└─$ date
echo "=== ORIGINAL ===" && sha256sum ~/Cases/OtterCTF2018/evidence/OtterCTF.vmem
echo "=== WORKING COPY ===" && sha256sum ~/Cases/OtterCTF2018/working/OtterCTF_working.vmem
Sun Apr 26 03:13:24 PM EEST 2026
=== ORIGINAL ===
5b3d8a9f9c96581a821c19b71dd6aa80dd299fc674b818f443f3a6fb5495a872 /home/mitm/Cases/OtterCTF2018/evidence/OtterCTF.vmem
=== WORKING COPY ===
5b3d8a9f9c96581a821c19b71dd6aa80dd299fc674b818f443f3a6fb5495a872 /home/mitm/Cases/OtterCTF2018/working/OtterCTF_working.vmem

```

Figure 3.6. SHA-256 hashes of the original evidence and the working copy match exactly, confirming the working copy is a bit-for-bit duplicate suitable for analysis.

```

└─$ date
ls -la ~/Cases/OtterCTF2018/evidence/OtterCTF.vmem
Sun Apr 26 03:21:57 PM EEST 2026
-r--r--r-- 1 mitm mitm 2147483648 Oct 24 2018 /home/mitm/Cases/OtterCTF2018/evidence/OtterCTF.vmem

```

Figure 3.7. The original `OtterCTF.vmem` file in the `evidence/` directory is set to read-only (`-r--r--r--`), preventing accidental modification during analysis.

3.7 Phase 0 Summary

Phase 0 was completed successfully. The analysis environment was isolated from production networks via host-only networking, the original evidence was hashed and write-protected, and a working copy was verified to be bit-for-bit identical. All required forensic tools were installed and version-verified. The investigation was now ready to proceed to Phase 1.

Table 3.2. Phase 0 completion summary

Step	Description	Status
0.1	Case folder structure created	Complete
0.2	Isolated VM and clean snapshot	Complete
0.3	All tools installed and verified	Complete
0.4	Original evidence hashed (MD5 + SHA-256)	Complete
0.5	Working copy verified, original set read-only	Complete

Chapter 4

Phase 1 — System Profiling

4.1 Overview

Phase 1 established the baseline context of the compromised system. Before deep malware analysis can begin, the investigator must answer fundamental questions: What operating system was running? Who was logged on? What hostname identifies this machine? What processes were active? What network connections existed at the moment of capture? Phase 1 produced seven evidence screenshots that collectively answer these questions.

4.2 Step 1.1 — Image Identification

The first task was to identify the operating system and architecture of the captured system. Volatility 2 was invoked with the `imageinfo` plugin, which scans for kernel debugging structures and suggests the most appropriate analysis profile.

```
cd ~/Cases/OtterCTF2018/working
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem imageinfo \
    | tee ../output/01_imageinfo.txt
```

Listing 4.1. Command: Volatility 2 imageinfo


```

└─$ cd ~/Cases/OtterCTF2018/working

date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem imageinfo \
    | tee ../output/01_imageinfo.txt
Sun Apr 26 04:13:29 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
INFO : volatility.debug : Determining profile based on KDBG search...
      Suggested Profile(s) : Win7SP1x64, Win7SP0x64, Win2008R2SP0x64, Win2008R2SP1x64_24000, Win2008R2SP1x64_23418, Win2008R2SP1x64, Win7SP1x64_24000, Win7SP1x64_23418
      AS Layer1 : WindowsAMD64PagedMemory (Kernel AS)
      AS Layer2 : FileAddressSpace (/home/mitm/Cases/OtterCTF2018/working/OtterCTF_working.vmem)
      PAE type : No PAE
      DTB : 0x187000L
      KDBG : 0xf80002c430a0L
      Number of Processors : 2
      Image Type (Service Pack) : 1
      KPCR for CPU 0 : 0xfffff80002c44d00L
      KPCR for CPU 1 : 0xfffff800009ef000L
      KUSER_SHARED_DATA : 0xfffff78000000000L
      Image date and time : 2018-08-04 19:34:22 UTC+0000
      Image local date and time : 2018-08-04 22:34:22 +0300

```

Figure 4.1. Volatility 2 imageinfo output identifying Win7SP1x64 as the optimal profile, with KDBG at 0xf80002c430a0L and image capture date of 2018-08-04 19:34:22 UTC.

A secondary verification was performed using Volatility 3, which uses a different symbol-based approach to confirm the operating system identification.

```

source ~/vol3-env/bin/activate
vol -f OtterCTF_working.vmem windows.info | tee ../output/01
    _vol3_info.txt

```

Listing 4.2. Command: Volatility 3 windows.info

```

---(vol3-env)-(mitm@kali):~/Cases/OtterCTF2018/working
└─$ source ~/vol3-env/bin/activate
vol -f OtterCTF_working.vmem windows.info | tee ../output/01_vol3_info.txt
WARNING: volatility3.framework.layers.vmem: No metadata file found alongside VMEM file. A VMSS or VMFS file may be required to correctly process a VMEM file. These should be placed in the same directory with the same file name, e.g. Ott
erCTF_working.vmem and OtterCTF_working.vmemss.
Volatility 3 Framework 2.27.0 - PDB scanning finished

Variable      Value
-----
Kernel Base   0xf80002a09000
DTB           0x187000
Symbols file: //home/mitm/vol3-env/lib/python3.13/site-packages/volatility3/symbols/windows/ntkrnlmp.pdf/384408B920174967BE7AA4A2C20A30FA-2.json.xz
Is64Bit       True
ISPAE         False
Layer_name    0 WindowsIntel32e
memory_layer  1 FileLayer
KdVersionBlock 0xf80002bfa068
MajorMinor    33836.65596
MachineType   19597
KdNumberProcessors 0
SystemTime    2018-08-04 19:34:22+00:00
NTSystemRoot  C:\Windows
NTProductType NTProductWinNT
NTMajorVersion 6
NTMinorVersion 1
PE MajorOperatingSystemVersion 6
PE MinorOperatingSystemVersion 1
PE Machine    34404
PE TimeDateStamp Sat Nov 20 13:00:25 2010

```

Figure 4.2. Volatility 3 confirms 64-bit Windows, NT version 6.1, with system time 2018-08-04 19:34:22 UTC and 2 processors. Both frameworks agree on the system identification.

System identification

Operating system: Windows 7 SP1 x64 (NT 6.1)

Profile (Vol2): Win7SP1x64

Image capture time: 2018-08-04 19:34:22 UTC

Local time: 2018-08-04 22:34:22 +03:00 (suggests GMT+3 / Israel time zone)

Processor count: 2

Memory size: 2 GB

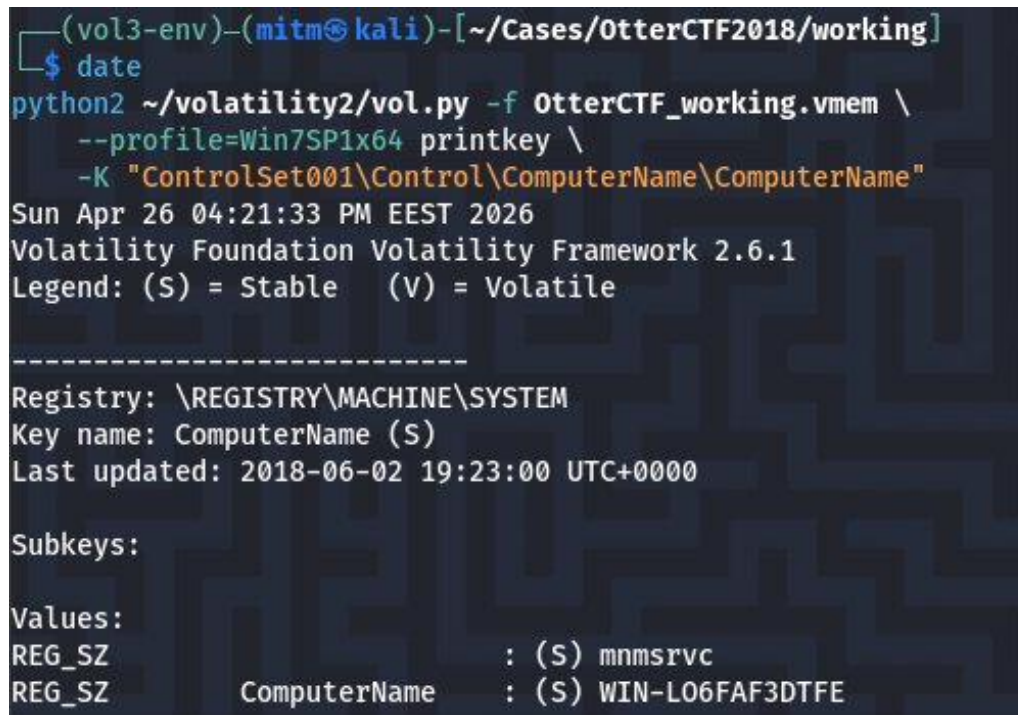
4.3 Step 1.2 — Hostname and User Context

Identifying the hostname locates this evidence in the broader environment. The Windows registry hive HKLM\SYSTEM\ControlSet001\Control\ComputerName\ComputerName stores this

value persistently.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 printkey \
  -K "ControlSet001\Control\ComputerName\ComputerName"
```

Listing 4.3. Command: hostname extraction



```
(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 printkey \
  -K "ControlSet001\Control\ComputerName\ComputerName"
Sun Apr 26 04:21:33 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
Legend: (S) = Stable (V) = Volatile

-----
Registry: \REGISTRY\MACHINE\SYSTEM
Key name: ComputerName (S)
Last updated: 2018-06-02 19:23:00 UTC+0000

Subkeys:

Values:
REG_SZ                               : (S) mnmsrvc
REG_SZ ComputerName                  : (S) WIN-L06FAF3DTFE
```

Figure 4.3. Registry value ComputerName = WIN-L06FAF3DTFE. The presence of the secondary value mnmsrvc (Net Meeting Remote Desktop Sharing) is a default Windows 7 system service.

The user context was then established by examining environment variables across all processes, filtered for the username and user profile path.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 envvars \
  | grep -iE "username|userprofile" \
  | grep -i "rick" \
  | head -20
```

Listing 4.4. Command: user context discovery


```
(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 envvars \
  | grep -iE "username|userprofile" \
  | grep -i "rick" \
  | head -20
Sun Apr 26 04:23:56 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
2344 taskhost.exe      0x0000000000201320 USERNAME      Rick
2344 taskhost.exe      0x0000000000201320 USERPROFILE    C:\Users\Rick
2704 dwm.exe           0x00000000001c1320 USERNAME      Rick
2704 dwm.exe           0x00000000001c1320 USERPROFILE    C:\Users\Rick
2728 explorer.exe      0x000000000025e1a90 USERNAME      Rick
2728 explorer.exe      0x000000000025e1a90 USERPROFILE    C:\Users\Rick
2804 vmtoolsd.exe       0x0000000000341320 USERNAME      Rick
2804 vmtoolsd.exe       0x0000000000341320 USERPROFILE    C:\Users\Rick
2836 BitTorrent.exe    0x00000000001f1320 USERNAME      Rick
2836 BitTorrent.exe    0x00000000001f1320 USERPROFILE    C:\Users\Rick
2308 bittorrentie.e     0x0000000000321320 USERNAME      Rick
2308 bittorrentie.e     0x0000000000321320 USERPROFILE    C:\Users\Rick
2624 bittorrentie.e     0x00000000000531320 USERNAME      Rick
2624 bittorrentie.e     0x00000000000531320 USERPROFILE    C:\Users\Rick
708 LunarMS.exe        0x000000000002f1320 USERNAME      Rick
708 LunarMS.exe        0x000000000002f1320 USERPROFILE    C:\Users\Rick
4076 chrome.exe        0x00000000005f61240 USERNAME      Rick
4076 chrome.exe        0x00000000005f61240 USERPROFILE    C:\Users\Rick
4084 chrome.exe        0x0000000000341320 USERNAME      Rick
4084 chrome.exe        0x0000000000341320 USERPROFILE    C:\Users\Rick
```

Figure 4.4. Environment variables across multiple processes consistently report USERNAME=Rick and USERPROFILE=C:\Users\Rick, confirming that Rick was the active user across the entire system at capture time.

System context

Hostname: WIN-L06FAF3DTFE

Active user: Rick

User profile path: C:\Users\Rick

Domain: Workgroup (no domain join detected)

4.4 Step 1.3 — Process Enumeration

Two complementary plugins were used to enumerate active processes: `pslist` (which walks the kernel's EPROCESS doubly-linked list) and `pstree` (which renders the parent-child relationships). The output was filtered to focus on processes of interest based on case context.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 pstree \
  | grep -A2 "explorer.exe\|Rick And Morty\|vmware-tray\|BitTorrent
\|LunarMS\|chrome.exe"
```

Listing 4.5. Command: process tree

```
(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 pstree \
  | grep -A2 "explorer.exe|Rick And Morty|vmware-tray|BitTorrent|LunarMS|chrome.exe"
Sun Apr 26 04:39:47 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
0xfffffa801b27e060:explorer.exe          2728    2696    33    854 2018-08-04 19:27:04 UTC+0000
. 0xfffffa801b486b30:Rick And Morty          3820    2728     4    185 2018-08-04 19:32:55 UTC+0000
.. 0xfffffa801a4c5b30:vmware-tray.exe        3720    3820     8    147 2018-08-04 19:33:02 UTC+0000
. 0xfffffa801b2f02e0:WebCompanion.e        2844    2728     0 ----- 2018-08-04 19:27:07 UTC+0000
. 0xfffffa801a4e3870:chrome.exe            4076    2728    44   1160 2018-08-04 19:29:30 UTC+0000
.. 0xfffffa801a4eab30:chrome.exe            4084    4076     8     86 2018-08-04 19:29:30 UTC+0000
.. 0xfffffa801a5ef1f0:chrome.exe            1796    4076    15    170 2018-08-04 19:33:41 UTC+0000
.. 0xfffffa801aa00a90:chrome.exe            3924    4076    16    228 2018-08-04 19:29:51 UTC+0000
.. 0xfffffa801a635240:chrome.exe            3648    4076    16    207 2018-08-04 19:33:38 UTC+0000
.. 0xfffffa801a502b30:chrome.exe             576    4076     2     58 2018-08-04 19:29:31 UTC+0000
.. 0xfffffa801a4f7b30:chrome.exe            1808    4076    13    229 2018-08-04 19:29:32 UTC+0000
.. 0xfffffa801a7f98f0:chrome.exe            2748    4076    15    181 2018-08-04 19:31:15 UTC+0000
. 0xfffffa801b5cb740:LunarMS.exe             708    2728    18    346 2018-08-04 19:27:39 UTC+0000
. 0xfffffa801b1cdb30:vmtoolsd.exe            2804    2728     6    190 2018-08-04 19:27:06 UTC+0000
. 0xfffffa801b290b30:BitTorrent.exe          2836    2728    24    471 2018-08-04 19:27:07 UTC+0000
.. 0xfffffa801b4c9b30:bittorrentie.e          2624    2836    13    316 2018-08-04 19:27:21 UTC+0000
.. 0xfffffa801b4a7b30:bittorrentie.e          2308    2836    15    337 2018-08-04 19:27:19 UTC+0000
```

Figure 4.5. Process tree showing parent-child relationships. Critical finding: Rick And Morty (PID 3820) spawned vmware-tray.exe (PID 3720), which is the ransomware payload. Both descend from explorer.exe (PID 2728), indicating user-initiated execution.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 pslist \
  | grep -iE "Rick|vmware-tray|BitTorrent|LunarMS|chrome"
```

Listing 4.6. Command: process list

```
(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 pslist \
  | grep -iE "Rick|vmware-tray|BitTorrent|LunarMS|chrome"
Sun Apr 26 04:40:10 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
0xfffffa801b290b30 BitTorrent.exe          2836    2728    24    471    1    1 2018-08-04 19:27:07 UTC+0000
0xfffffa801b4a7b30 bittorrentie.e          2308    2836    15    337    1    1 2018-08-04 19:27:19 UTC+0000
0xfffffa801b4c9b30 bittorrentie.e          2624    2836    13    316    1    1 2018-08-04 19:27:21 UTC+0000
0xfffffa801b5cb740 LunarMS.exe             708    2728    18    346    1    1 2018-08-04 19:27:39 UTC+0000
0xfffffa801a4e3870 chrome.exe            4076    2728    44   1160    1    0 2018-08-04 19:29:30 UTC+0000
0xfffffa801a4eab30 chrome.exe            4084    4076     8     86    1    0 2018-08-04 19:29:30 UTC+0000
0xfffffa801a502b30 chrome.exe             576    4076     2     58    1    0 2018-08-04 19:29:31 UTC+0000
0xfffffa801a4f7b30 chrome.exe            1808    4076    13    229    1    0 2018-08-04 19:29:32 UTC+0000
0xfffffa801aa00a90 chrome.exe            3924    4076    16    228    1    0 2018-08-04 19:29:51 UTC+0000
0xfffffa801a7f98f0 chrome.exe            2748    4076    15    181    1    0 2018-08-04 19:31:15 UTC+0000
0xfffffa801b486b30 Rick And Morty          3820    2728     4    185    1    1 2018-08-04 19:32:55 UTC+0000
0xfffffa801a4c5b30 vmware-tray.exe        3720    3820     8    147    1    1 2018-08-04 19:33:02 UTC+0000
0xfffffa801a635240 chrome.exe            3648    4076    16    207    1    0 2018-08-04 19:33:38 UTC+0000
0xfffffa801a5ef1f0 chrome.exe            1796    4076    15    170    1    0 2018-08-04 19:33:41 UTC+0000
```

Figure 4.6. Filtered pslist output showing all relevant processes with their start times. The chronological ordering reveals the attack sequence: BitTorrent (19:27:07), chrome (19:29:30), Rick And Morty (19:32:55), and vmware-tray (19:33:02).

Critical processes identified

Process	PID	Parent	Start time (UTC)
explorer.exe	2728	2696	19:27:04
BitTorrent.exe	2836	2728	19:27:07
LunarMS.exe	708	2728	19:27:39
chrome.exe	4076	2728	19:29:30
Rick And Morty	3820	2728	19:32:55
vmware-tray.ex	3720	3820	19:33:02

The process tree provides the first piece of strong evidence: `vmware-tray.ex` (PID 3720) is a child of Rick And Morty (PID 3820), which strongly suggests the so-called Rick And Morty download was actually a dropper that spawned the suspicious `vmware-tray` executable.

4.5 Step 1.4 — Network Connections

Active and recent network connections were enumerated using the `netscan` plugin, which scans memory for `TCP_ENDPOINT` and `UDP_ENDPOINT` structures. The output was filtered for processes of interest.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 netscan \
  | grep -iE "LunarMS|vmware|Rick|77.102"
```

Listing 4.7. Command: network scan

```
(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 netscan \
  | grep -iE "LunarMS|vmware|Rick|77.102"
Sun Apr 26 04:41:45 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
0x7d6124d0    TCPv4    192.168.202.131:49530    77.102.199.102:7575    CLOSED    708    LunarMS.exe
0x7e413a40    TCPv4    -:0                    -:0                    CLOSED    708    LunarMS.exe
0x7e521b50    TCPv4    -:0                    -:0                    CLOSED    708    LunarMS.exe
```

Figure 4.7. LunarMS.exe (PID 708) maintained an outbound TCP connection from 192.168.202.131:49530 to 77.102.199.102:7575. The state is `CLOSED`, indicating the connection had been torn down by the time of capture, but the endpoint structure remained in memory.

Network indicators of compromise

Victim IP address: 192.168.202.131

Suspicious external endpoint: 77.102.199.102:7575

Owning process: LunarMS.exe (PID 708)

Significance: Port 7575 is non-standard for any commercial gaming protocol. The pairing of an unusual port with a non-Microsoft hosted IP is consistent with a custom command-and-control (C2) channel.

4.6 Phase 1 Summary

Phase 1 successfully established the baseline system context. The compromised host runs Windows 7 SP1 x64, has hostname WIN-L06FAF3DTFE, and was being used by user Rick at the time of capture. The process tree reveals a suspicious parent-child relationship between Rick And Morty (apparent dropper) and vmware-tray.exe (apparent payload). A C2 connection to 77.102.199.102:7575 was active in the recent past, owned by LunarMS.exe.

Table 4.1. Phase 1 completion summary

Step	Description	Status
1.1	OS identification (Win7SP1x64 confirmed)	Complete
1.2	Hostname and user context recovered	Complete
1.3	Critical processes enumerated and ranked	Complete
1.4	C2 network connection identified	Complete

Chapter 5

Phase 2 — Attack Vector Identification

5.1 Overview

Phase 2 traced the infection backwards from the suspicious `vmware-tray.exe` process discovered in Phase 1. The objective was to answer: where did this binary come from, how did it arrive on the system, and what user actions led to its execution? This was achieved by examining process command lines, scanning the file system memory artifacts, extracting the Chrome browser history database from process memory, and computing hashes for VirusTotal lookup. Phase 2 produced seven evidence screenshots.

5.2 Step 2.1 — Suspicious Process Command Line

The `cmdline` plugin extracts the full command line from the `PEB.ProcessParameters` structure, revealing exactly how each process was invoked.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 cmdline \
    | grep -iE "vmware|rick|morty|temp|torrent|notepad" \
    --after-context=2
```

Listing 5.1. Command: command-line extraction

```

--(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 cmdline \
| grep -iE "vmware|rick|morty|temp|torrent|notepad" \
--after-context=2
Sun Apr 26 05:18:44 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
Command line : "C:\Program Files\VMware\VMware Tools\vmacthlp.exe"
svchost.exe pid: 712
Command line : "C:\Program Files\VMware\VMware Tools\VMware VGAuthService.exe"
vmtoolsd.exe pid: 1428
Command line : "C:\Program Files\VMware\VMware Tools\vmtoolsd.exe"
WmiPrvSE.exe pid: 1800
Command line : "C:\Program Files\VMware\VMware Tools\vmtoolsd.exe" -n vmusr
BitTorrent.exe pid: 2836
Command line : "C:\Users\Rick\AppData\Roaming\BitTorrent\BitTorrent.exe" /MINIMIZED
WebCompanion.exe pid: 2844
bittorrentie.exe pid: 2308
Command line : "C:\Users\Rick\AppData\Roaming\BitTorrent\updates\7.10.3_44495\bittorrentie.exe" BitTorrent_2836_00313D08_590648902 BT4823DF041B09 BitTorrent
bittorrentie.exe pid: 2624
Command line : "C:\Users\Rick\AppData\Roaming\BitTorrent\updates\7.10.3_44495\bittorrentie.exe" BitTorrent_2836_00313978_1933444659 BT4823DF041B09 BitTorrent
LunarMS.exe pid: 708
Rick And Morty pid: 3820
Command line : "C:\Torrents\Rick And Morty season 1 download.exe"
vmware-tray.exe pid: 3720
Command line : "C:\Users\Rick\AppData\Local\Temp\RarSFX0\vmware-tray.exe"
WebCompanionIn pid: 3880
notepad.exe pid: 3304
Command line : "C:\Windows\system32\NOTEPAD.EXE" C:\Users\Rick\Desktop\Flag.txt.WINDOWS
cmd.exe pid: 3916

```

Figure 5.1. Command-line evidence reveals the full attack path. The dropper at C:\Torrents\Rick And Morty season 1 download.exe (PID 3820) spawned the ransomware at C:\Users\Rick\AppData\Local\Temp\RarSFX0\vmware-tray.exe (PID 3720). The RarSFX0 folder name confirms the dropper is a RAR self-extracting archive.

Initial dropper and payload paths

Dropper location: C:\Torrents\Rick And Morty season 1 download.exe
Payload location: C:\Users\Rick\AppData\Local\Temp\RarSFX0\vmware-tray.exe
Notepad.exe argument: C:\Users\Rick\Desktop\Flag.txt.WINDOWS
Inferred ransomware extension: .WINDOWS

The Flag.txt.WINDOWS argument to Notepad is particularly significant: this is the ransom note file, and the .WINDOWS extension is the marker the ransomware appends to encrypted files. This single piece of evidence confirms the file encryption.

5.3 Step 2.2 — Chrome History File Discovery

Browser history was recovered from process memory by first locating the Chrome History SQLite database in the file scan output. The filescan plugin enumerates all `_FILE_OBJECT` structures in memory.

```

date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 filescan \
| grep -iE "chrome.*history|history.*chrome" \
| tee ../output/06_chrome_filescan.txt

```

Listing 5.2. Command: Chrome history file scan

```

--(vol3-env)--(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 filescan \
| grep -iE "chrome.*history|history.*chrome" \
| tee ../output/06_chrome_filescan.txt
Sun Apr 26 05:21:30 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
0x000000007d45dcc0 18 1 RW-rw- \Device\HarddiskVolume1\Users\Rick\AppData\Local\Google\Chrome\User Data\Default\History
0x000000007dac7410 33 1 RW-rw- \Device\HarddiskVolume1\Users\Rick\AppData\Local\Google\Chrome\User Data\Default\History-journal

```

Figure 5.2. Two file objects located: the main History database at offset 0x000000007d45dcc0 and the rollback journal History-journal at 0x000000007dac7410. Both reside in Rick's Chrome user data directory.

5.4 Step 2.3 — Chrome History Extraction

The Chrome History file object was extracted from memory using the `dumpfiles` plugin with the offset identified in step 2.2.

```

mkdir -p ../output/chrome_dump
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 dumpfiles \
-Q 0x000000007d45dcc0 \
-D ../output/chrome_dump/ --name

ls -lah ../output/chrome_dump/

```

Listing 5.3. Command: Chrome history extraction

```

--(vol3-env)--(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ mkdir -p ../output/chrome_dump
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 dumpfiles \
-Q 0x000000007d45dcc0 \
-D ../output/chrome_dump/ \
--name

ls -lah ../output/chrome_dump/
Sun Apr 26 05:23:55 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
DataSectionObject 0x7d45dcc0 None \Device\HarddiskVolume1\Users\Rick\AppData\Local\Google\Chrome\User Data\Default\History
SharedCacheMap 0x7d45dcc0 None \Device\HarddiskVolume1\Users\Rick\AppData\Local\Google\Chrome\User Data\Default\History
total 200K
drwxrwxr-x 2 mitm mitm 4.0K Apr 26 17:23 .
drwxrwxr-x 3 mitm mitm 4.0K Apr 26 17:23 ..
-rw-rw-r-- 1 mitm mitm 0 Apr 26 17:23 file.None.0xfffffa801a4ec470.History.vacb
-rw-rw-r-- 1 mitm mitm 192K Apr 26 17:23 file.None.0xfffffa801a5193d0.History.dat

```

Figure 5.3. The 192 KB SQLite database was successfully extracted to file.None.0xfffffa801a5193d0.History.dat, ready for SQL querying.

5.5 Step 2.4 — Browser History SQL Query

The extracted SQLite database was queried for URLs related to torrenting and to the MEGA file-sharing service. The Chrome timestamp format (microseconds since 1601) was converted to standard Unix epoch for readability.

```
sqlite3 ../output/chrome_dump/file.None.0xfffffa801a5193d0.History.
dat \
"SELECT url, title, visit_count, \
datetime(last_visit_time/1000000-11644473600,'unixepoch') AS
visit_time \
FROM urls WHERE url LIKE '%thepiratebay%' OR url LIKE '%torrent%' \
ORDER BY last_visit_time DESC;"
```

Listing 5.4. Query: torrent search history

```
(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
sqlite3 ../output/chrome_dump/file.None.0xfffffa801a5193d0.History.dat \
"SELECT url, title, visit_count, \
datetime(last_visit_time/1000000,'unixepoch') AS visit_time \
FROM urls WHERE url LIKE '%thepiratebay%' OR url LIKE '%torrent%' \
ORDER BY last_visit_time DESC;"
Sun Apr 26 05:26:41 PM EEST 2026
https://thepiratebay3.org/?url=https://tpbship.org/torrent/12162845/Rick_and_Morty_Season_1_[1080p]_[HEVC]|Rick and Morty Season 1 [1080p] [H
EVC] (download torrent) - TPB|1|2018-07-07 20:19:28
https://thepiratebay3.org/index.php?q=rick+and+morty|ThePirateBay - Download music, movies, games, software! The Pirate Bay - The galaxy's mo
st resilient BitTorrent site|3|2018-07-07 18:38:27
https://thepiratebay3.org/?url=https://tpbship.org/torrent/12678054/Rick_and_Morty_Season_2_[WEBRIP]_[1080p]_[HEVC]|Rick and Morty Season 2 [
WEBRIP] [1080p] [HEVC] (download torrent) - TPB|1|2018-07-07 18:38:13
http://bittorrent.com/prodnews?uid=ue1-a70900fae7df446396e987ac011e0cdf8v=7.10.3.1.44495|BitTorrent - Product News|1|2018-07-07 18:37:26
http://www.bittorrent.com/prodnews?uid=ue1-a70900fae7df446396e987ac011e0cdf8v=7.10.3.1.44495|BitTorrent - Product News|1|2018-07-07 18:37:26
https://www.bittorrent.com/prodnews?uid=ue1-a70900fae7df446396e987ac011e0cdf8v=7.10.3.1.44495|BitTorrent - Product News|1|2018-07-07 18:37:26
http://cdn.bitmedianetwork.com/network/r.html?u=ue1-a70900fae7df446396e987ac011e0cdf8next=http://bittorrent.com/prodnews8osv=1DB101066iev=88g
eo=US6lang=en5ver=7%2e10%2e3%2e1%2e44495|BitTorrent - Product News|1|2018-07-07 18:37:22
https://thepiratebay3.org/?url=https://tpbship.org/torrent/18676863/Rick.and.Morty.Season.3.Complete.1080p.Uncensored.x264-Rapta|Rick.and.Mor
ty.Season.3.Complete.1080p.Uncensored.x264-Rapta (download torrent) - TPB|1|2018-07-07 18:36:14
https://thepiratebay3.org/|ThePirateBay - Download music, movies, games, software! The Pirate Bay - The galaxy's most resilient BitTorrent si
te|1|2018-07-07 18:35:12
https://www.google.co.il/search?q=thepiratebay&qq=thepiratebay&qqs=chrome..69i57j0l4j5.7503j0j7&sourceid=chrome&ie=UTF-8|thepiratebay - ב י פ ו
n -Google|1|2018-07-07 18:35:10
https://www.bittorrent.com/downloads/complete/track/stable/os/win/Download - BitTorrent® - Delivering the World's Content|1|2018-07-05 19:09:
29
https://www.bittorrent.com/downloads/win/Download - BitTorrent® - Delivering the World's Content|1|2018-07-05 19:09:25
https://www.bittorrent.com/downloads/Download - BitTorrent® - Delivering the World's Content|1|2018-07-05 19:09:25
https://www.google.co.il/search?q=bittorrent&qq=bittorre&qqs=chrome.1.69i57j0l5.4298j0j46&sourceid=chrome&ie=UTF-8|bittorrent - ב י פ ו n-Google
|1|2018-07-05 19:09:22
```

Figure 5.4. Chrome history shows Rick searching ThePirateBay for “rick and morty” on 2018-07-07 18:38:27, viewing the Season 1 torrent listing at 20:19:28. He also installed BitTorrent two days earlier on 2018-07-05.

```
sqlite3 ../output/chrome_dump/file.None.0xfffffa801a5193d0.History.
dat \
"SELECT url, title, visit_count, \
datetime(last_visit_time/1000000-11644473600,'unixepoch') AS
visit_time \
FROM urls WHERE url LIKE '%mega%' OR url LIKE '%lunarms%' \
ORDER BY last_visit_time DESC;"
```

Listing 5.5. Query: MEGA download history


```

--(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
└─$ date
sqlite3 ../output/chrome_dump/file.None.0xfffffa801a5193d0.History.dat \
"SELECT url, title, visit_count, \
datetime(last_visit_time/1000000-11644473600,'unixepoch') AS visit_time \
FROM urls WHERE url LIKE '%mega%' OR url LIKE '%lunarms%' \
ORDER BY last_visit_time DESC;"
Sun Apr 26 05:27:12 PM EEST 2026
https://mega.nz/#!iqYnmDjK!9oR81J9zsZsfbn28L7L7549hYqici_XuZCL4DNffcm|MEGA 21%|3|2018-07-07 20:07:45
http://lunarms.zapto.org/?base=main&page=download|LunarMS|2|2018-07-07 20:05:16
https://mega.nz/#!098x2RII!m6aWPFtXcycUczG6iaKxyuqHdKae391mmuNQfDWOcubY|MEGA|2|2018-07-07 18:34:46
http://lunarms.zapto.org/?base=ucp&page=characters|LunarMS|1|2018-07-07 18:34:33
http://lunarms.zapto.org/?base=ucp|LunarMS|1|2018-07-07 18:34:16
http://lunarms.zapto.org/?base=main&page=register|LunarMS|4|2018-07-07 18:34:16
http://lunarms.zapto.org/?base=main|LunarMS|1|2018-07-07 18:30:31
http://www.gtop100.com/topsites/MapleStory/sitedetails/LunarMS-v83-2000x-500x-2x-ONLINE-NOW-90563|LunarMS - v83- 2000x 500x 2x - ONLINE NOW - id-90563|1|2018-07-07 18:29:46

```

Figure 5.5. Chrome history reveals the dropper download path: Rick clicked a MEGA.nz link at 2018-07-07 20:07:45. He had also been visiting lunarms.zapto.org (the LunarMS game site) earlier the same day, registering an account at 18:34:16.

5.6 Step 2.5 — Dropper File Recovery

The dropper file itself was located in memory using `filesnscan` and extracted with `dumpfiles`.

```

date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 filesnscan \
| grep -iE "rick.*morty|morty.*rick|torrents|\.torrent" \
| tee ../output/08_dropper_filesnscan.txt

```

Listing 5.6. Command: dropper file scan

```

--(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
└─$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 filesnscan \
| grep -iE "rick.*morty|morty.*rick|torrents|\.torrent" \
| tee ../output/08_dropper_filesnscan.txt

cat ../output/08_dropper_filesnscan.txt
Sun Apr 26 05:27:43 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
0x000000007d63dbc0 10 0 R--r-d \Device\HarddiskVolume1\Torrents\Rick And Morty season 1 download.exe
0x000000007d6b3a10 11 1 R--rw- \Device\HarddiskVolume1\Torrents\Rick and Morty - Season 3 (2017) [1080p]\Rick.and.Morty.S03E07.The.Ricklantis.Mixup.1080
p.Amazon.WEB-DL.x264-Rapta.mkv
0x000000007d7adb50 17 1 R--rw- \Device\HarddiskVolume1\Torrents\Rick and Morty - Season 3 (2017) [1080p]\Rick.and.Morty.S03E06.Rest.and.Ricklaxation.108
p.Amazon.WEB-DL.x264-Rapta.mkv
0x000000007d7c1730 2 1 R--rwd \Device\HarddiskVolume1\Torrents
0x000000007d8813c0 2 0 RW-rwd \Device\HarddiskVolume1\Users\Rick\Downloads\Rick And Morty season 1 download.exe.torrent
0x000000007da56240 2 0 RW-rwd \Device\HarddiskVolume1\Torrents\Rick And Morty season 1 download.exe
0x000000007dae9350 2 0 RWD--- \Device\HarddiskVolume1\Users\Rick\AppData\Roaming\BitTorrent\Rick And Morty season 1 download.exe.1.torrent
0x000000007dcbf6f0 2 0 RW-rwd \Device\HarddiskVolume1\Users\Rick\AppData\Roaming\BitTorrent\Rick And Morty season 1 download.exe.1.torrent
0x000000007e40aab0 2 1 R--rwd \Device\HarddiskVolume1\Torrents
0x000000007e5f5d10 3 1 R--rw- \Device\HarddiskVolume1\Torrents\Rick and Morty Season 2 [WEBRIP] [1080p] [HEVC]\[pseudo] Rick and Morty S02E03 Auto Erot
ic Assimilation [1080p] [h.265].mkv
0x000000007e710070 8 0 R--rwd \Device\HarddiskVolume1\Torrents\Rick And Morty season 1 download.exe
0x000000007e7ae700 3 1 R--rw- \Device\HarddiskVolume1\Torrents\Rick and Morty Season 2 [WEBRIP] [1080p] [HEVC]\Sample\Screenshot 08.png
0x000000007d63dbc0 10 0 R--r-d \Device\HarddiskVolume1\Torrents\Rick And Morty season 1 download.exe
0x000000007d6b3a10 11 1 R--rw- \Device\HarddiskVolume1\Torrents\Rick and Morty - Season 3 (2017) [1080p]\Rick.and.Morty.S03E07.The.Ricklantis.Mixup.1080
p.Amazon.WEB-DL.x264-Rapta.mkv
0x000000007d7adb50 17 1 R--rw- \Device\HarddiskVolume1\Torrents\Rick and Morty - Season 3 (2017) [1080p]\Rick.and.Morty.S03E06.Rest.and.Ricklaxation.108
p.Amazon.WEB-DL.x264-Rapta.mkv
0x000000007d7c1730 2 1 R--rwd \Device\HarddiskVolume1\Torrents
0x000000007d8813c0 2 0 RW-rwd \Device\HarddiskVolume1\Users\Rick\Downloads\Rick And Morty season 1 download.exe.torrent
0x000000007da56240 2 0 RW-rwd \Device\HarddiskVolume1\Torrents\Rick And Morty season 1 download.exe
0x000000007dae9350 2 0 RWD--- \Device\HarddiskVolume1\Users\Rick\AppData\Roaming\BitTorrent\Rick And Morty season 1 download.exe.1.torrent
0x000000007dcbf6f0 2 0 RW-rwd \Device\HarddiskVolume1\Users\Rick\AppData\Roaming\BitTorrent\Rick And Morty season 1 download.exe.1.torrent
0x000000007e40aab0 2 1 R--rwd \Device\HarddiskVolume1\Torrents
0x000000007e5f5d10 3 1 R--rw- \Device\HarddiskVolume1\Torrents\Rick and Morty Season 2 [WEBRIP] [1080p] [HEVC]\[pseudo] Rick and Morty S02E03 Auto Erot
ic Assimilation [1080p] [h.265].mkv
0x000000007e710070 8 0 R--rwd \Device\HarddiskVolume1\Torrents\Rick And Morty season 1 download.exe
0x000000007e7ae700 3 1 R--rw- \Device\HarddiskVolume1\Torrents\Rick and Morty Season 2 [WEBRIP] [1080p] [HEVC]\Sample\Screenshot 08.png

```

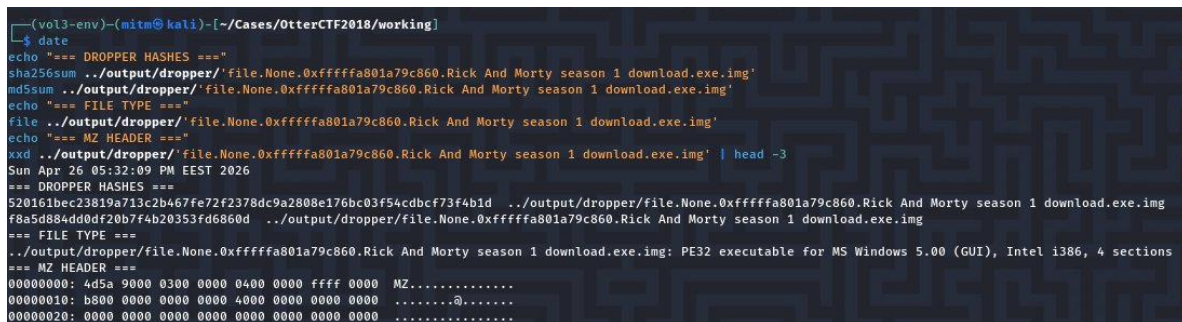
Figure 5.6. File scan reveals the dropper at offset 0x7d63dbc0 (Rick And Morty season 1 download.exe) located in \Torrents\, plus its associated .torrent metadata file at offset 0x7c1730. Multiple Rick and Morty media files indicate this was an active torrent download session.

5.7 Step 2.6 — Dropper Hashing

Both MD5 and SHA-256 hashes of the extracted dropper file were computed for VirusTotal lookup. The first 32 bytes were also examined to confirm the file format.

```
date
echo "=== DROPPER HASHES ==="
sha256sum './output/dropper/file.None.0xffffffffa801a79c860.Rick And
Morty season 1 download.exe.img'
md5sum './output/dropper/file.None.0xffffffffa801a79c860.Rick And
Morty season 1 download.exe.img'
echo "=== FILE TYPE ==="
file './output/dropper/file.None.0xffffffffa801a79c860.Rick And
Morty season 1 download.exe.img'
echo "=== MZ HEADER ==="
xxd './output/dropper/file.None.0xffffffffa801a79c860.Rick And
Morty season 1 download.exe.img' | head -3
```

Listing 5.7. Command: dropper hashing and signature check



```
(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
echo "=== DROPPER HASHES ==="
sha256sum './output/dropper/file.None.0xffffffffa801a79c860.Rick And Morty season 1 download.exe.img'
md5sum './output/dropper/file.None.0xffffffffa801a79c860.Rick And Morty season 1 download.exe.img'
echo "=== FILE TYPE ==="
file './output/dropper/file.None.0xffffffffa801a79c860.Rick And Morty season 1 download.exe.img'
echo "=== MZ HEADER ==="
xxd './output/dropper/file.None.0xffffffffa801a79c860.Rick And Morty season 1 download.exe.img' | head -3
Sun Apr 26 05:32:09 PM EEST 2026
=== DROPPER HASHES ===
520161bec23819a713c2b467fe72f2378dc9a2808e176bc03f54cdbcf73f4b1d ./output/dropper/file.None.0xffffffffa801a79c860.Rick And Morty season 1 download.exe.img
f8a5d884dd0df20b7f4b20353fd6860d ./output/dropper/file.None.0xffffffffa801a79c860.Rick And Morty season 1 download.exe.img
=== FILE TYPE ===
./output/dropper/file.None.0xffffffffa801a79c860.Rick And Morty season 1 download.exe.img: PE32 executable for MS Windows 5.00 (GUI), Intel i386, 4 sections
=== MZ HEADER ===
00000000: 4d5a 9000 0300 0000 0400 0000 ffff 0000  MZ.....
00000010: b800 0000 0000 0000 4000 0000 0000 0000  .....@.....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  .....
```

Figure 5.7. The dropper is a PE32 executable for MS Windows 5.00 GUI, Intel i386, 4 sections. SHA-256: 520161bec23819a713c2b467fe72f2378dc9a2808e176bc03f54cdbcf73f4b1d. The 4D 5A (MZ) magic bytes confirm a valid Windows PE file.

Dropper indicators of compromise (IOC)

Filename: Rick And Morty season 1 download.exe
File type: PE32 executable, MS Windows GUI, Intel i386
Architecture: 32-bit (despite running on a 64-bit OS)
SHA-256: 520161bec23819a713c2b467fe72f2378dc9a2808e176bc03f54cdbcf73f4b1d
MD5: f8a5d884dd0df20b7f4b20353fd6860d

5.8 Step 2.7 — VirusTotal Hash Lookup

The dropper SHA-256 hash was submitted to VirusTotal for community antivirus reputation lookup.

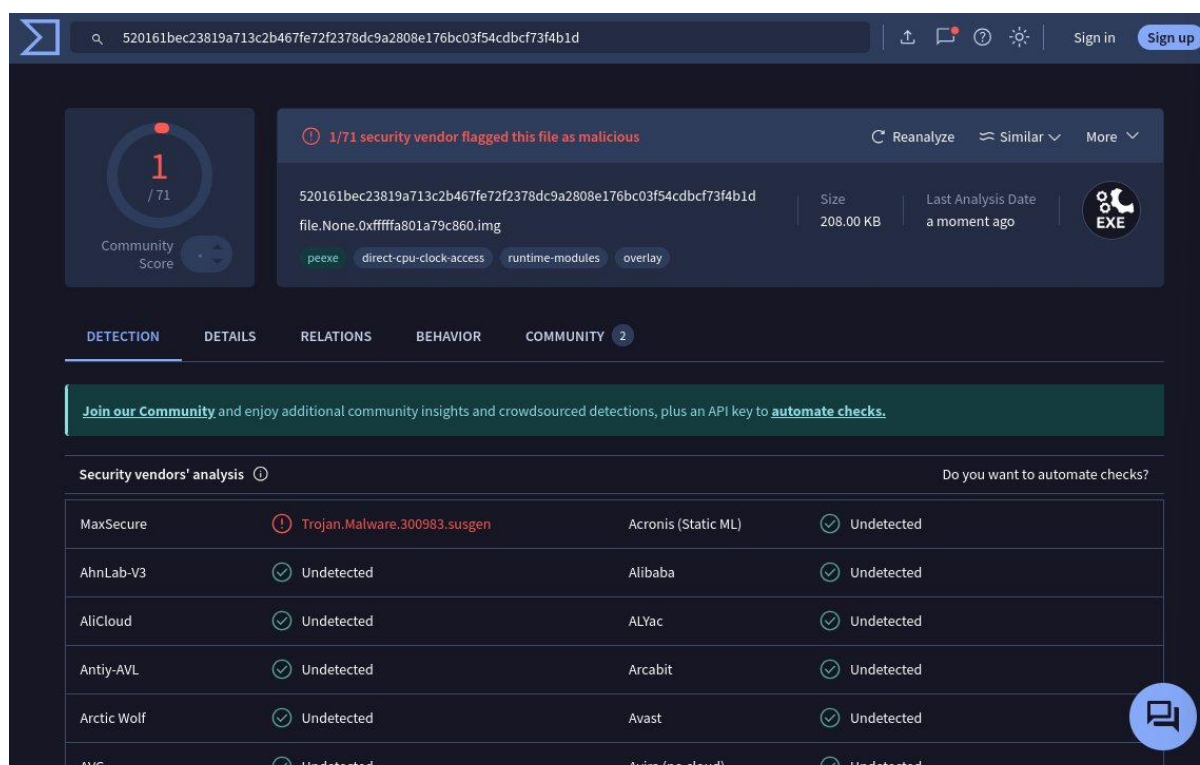


Figure 5.8. VirusTotal reports 1 of 71 antivirus engines flagged this file as malicious (MaxSecure: Trojan.Malware.300983.susgen). The detection ratio of 1/71 highlights why traditional signature-based AV is insufficient against custom or low-distribution malware — this file would have been delivered without warning.

Forensic significance

A 1/71 VirusTotal detection rate is consistent with a low-prevalence custom malware build. From a defensive perspective, this confirms that endpoint AV alone would not have prevented this attack. Proactive controls — application allowlisting, egress filtering of P2P, and user awareness — are required.

5.9 Phase 2 Summary

Phase 2 fully established the attack vector. The infection began on July 7, 2018 when Rick searched for pirated Rick and Morty content on ThePirateBay. He clicked a torrent that linked to a MEGA.nz file. The torrent was downloaded via BitTorrent and saved to `C:\Torrents\Rick And Morty season 1 download.exe`. Twenty-eight days later, on August 4, 2018, Rick double-clicked this file. It was a RAR self-extracting archive (RarSFX0) that dropped `vmware-tray.exe` into `%TEMP%` and launched it. VirusTotal confirms only 1 of 71 AV engines would have flagged this file at the time.

Table 5.1. Phase 2 completion summary

Step	Description	Status
2.1	Command-line evidence (dropper → payload chain)	Complete
2.2	Chrome history file located in memory	Complete
2.3	Chrome SQLite database extracted	Complete
2.4	Torrent and MEGA URLs recovered from history	Complete
2.5	Dropper located via filescan and extracted	Complete
2.6	Dropper hashed (SHA-256, MD5)	Complete
2.7	VirusTotal lookup (1/71 detection)	Complete

Chapter 6

Phase 3 — Malware Analysis

6.1 Overview

Phase 3 was the deepest and most technically demanding stage of the investigation. The objective was to perform a complete forensic autopsy of the ransomware process `vmware-tray.exe` (PID 3720): detect injected code regions, identify hidden libraries, dump the process memory, recover encryption keys and ransom note text, extract and crack credential hashes, and document persistence (or its absence). Phase 3 produced sixteen evidence screenshots, the largest of any phase.

6.2 Step 3.1 — Code Injection Detection (malfind)

The Volatility `malfind` plugin scans process memory for regions marked `PAGE_EXECUTE_READWRITE` (RWX), which are highly suspicious because legitimate code rarely needs simultaneous write and execute permissions. The plugin was targeted at PID 3720.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 malfind \
    --pid 3720
```

Listing 6.1. Command: malfind injection scan


```

(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
└─$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 malfind \
  --pid 3720
Sun Apr 26 11:44:09 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
Process: vmware-tray.ex Pid: 3720 Address: 0x670000
Vad Tag: VadS Protection: PAGE_EXECUTE_READWRITE
Flags: CommitCharge: 1, PrivateMemory: 1, Protection: 6

0x0000000000670000  9c 4c 5b ae 8c 8a 00 01 ee ff ee ff 00 00 00 00  .L[.....
0x0000000000670010  a8 00 67 00 a8 00 67 00 00 00 67 00 00 00 67 00  ..g...g...g...g.
0x0000000000670020  40 00 00 00 88 05 67 00 00 00 6b 00 3f 00 00 00  @.....g...k?...
0x0000000000670030  01 00 00 00 00 00 00 00 f0 0f 67 00 f0 0f 67 00  .....g...g.

0x0000000000670000  9c          PUSHF
0x0000000000670001  4c          DEC ESP
0x0000000000670002  5b          POP EBX
0x0000000000670003  ae          SCASB
0x0000000000670004  8c8a0001eeff MOV [EDX-0x11ff00], CS
0x000000000067000a  ee          OUT DX, AL
0x000000000067000b  ff00        INC DWORD [EAX]
0x000000000067000d  0000        ADD [EAX], AL
0x000000000067000f  00a8006700a8 ADD [EAX-0x57ff9900], CH
0x0000000000670015  006700      ADD [EDI+0x0], AH
0x0000000000670018  0000        ADD [EAX], AL
0x000000000067001a  670000      ADD [BX+SI], AL
0x000000000067001d  006700      ADD [EDI+0x0], AH
0x0000000000670020  40          INC EAX
0x0000000000670021  0000        ADD [EAX], AL
0x0000000000670023  008805670000 ADD [EAX+0x6705], CL
0x0000000000670029  006b00      ADD [EBX+0x0], CH
0x000000000067002c  3f          AAS
0x000000000067002d  0000        ADD [EAX], AL
0x000000000067002f  0001        ADD [ECX], AL
0x0000000000670031  0000        ADD [EAX], AL
0x0000000000670033  0000        ADD [EAX], AL
0x0000000000670035  0000        ADD [EAX], AL
0x0000000000670037  00f0        ADD AL, DH
0x0000000000670039  0f6700      PACKUSWB MM0, [EAX]
0x000000000067003c  f0f6700     PACKUSWB MM0, [EAX]

```

Figure 6.1. First injected memory region at 0x670000 in vmware-tray.ex (PID 3720). Vad Tag: VadS, Protection: PAGE_EXECUTE_READWRITE, CommitCharge: 1, PrivateMemory: 1. The hex dump shows no MZ (4D 5A) header, indicating raw shellcode rather than a full PE file.

```

(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
└─$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 malfind \
  --pid 3720 \
  | grep -A 35 "Address: 0x510000"
Sun Apr 26 11:45:27 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
Process: vmware-tray.exe Pid: 3720 Address: 0x510000
Vad Tag: VadS Protection: PAGE_EXECUTE_READWRITE
Flags: CommitCharge: 23, PrivateMemory: 1, Protection: 6

0x0000000000051000  f0 26 16 8e 6a 2f 00 01 ee ff ee ff 00 00 00 00  .8..j/.....
0x0000000000051010  a8 00 51 00 a8 00 51 00 00 00 51 00 00 00 51 00  ..Q...Q...Q...Q.
0x0000000000051020  40 00 00 00 88 05 51 00 00 00 55 00 29 00 00 00  @.....Q...U.)...
0x0000000000051030  01 00 00 00 00 00 00 00 f0 6f 52 00 f0 6f 52 00  .....oR...oR..

0x0000000000051000  f02616          PUSH SS
0x0000000000051003  8e6a2f          MOV GS, [EDX+0x2f]
0x0000000000051006  0001           ADD [ECX], AL
0x0000000000051008  ee            OUT DX, AL
0x0000000000051009  ff            DB 0xff
0x000000000005100a  ee            OUT DX, AL
0x000000000005100b  ff00          INC DWORD [EAX]
0x000000000005100d  0000          ADD [EAX], AL
0x000000000005100f  00a8005100a8  ADD [EAX-0x57ffaf00], CH
0x0000000000051015  005100        ADD [ECX+0x0], DL
0x0000000000051018  0000          ADD [EAX], AL
0x000000000005101a  51            PUSH ECX
0x000000000005101b  0000          ADD [EAX], AL
0x000000000005101d  005100        ADD [ECX+0x0], DL
0x0000000000051020  40            INC EAX
0x0000000000051021  0000          ADD [EAX], AL
0x0000000000051023  008805510000  ADD [EAX+0x5105], CL
0x0000000000051029  005500        ADD [EBP+0x0], DL
0x000000000005102c  2900          SUB [EAX], EAX
0x000000000005102e  0000          ADD [EAX], AL
0x0000000000051030  0100          ADD [EAX], EAX
0x0000000000051032  0000          ADD [EAX], AL
0x0000000000051034  0000          ADD [EAX], AL
0x0000000000051036  0000          ADD [EAX], AL
0x0000000000051038  f06f          OUTS DX, DWORD [ESI]
0x000000000005103a  52            PUSH EDX
0x000000000005103b  00f0          ADD AL, DH

```

Figure 6.2. Largest injected region at 0x510000, CommitCharge: 23 (23 committed pages = approximately 92 KB of injected code). The disassembly is consistent with stub or trampoline code rather than a structured executable.

In total, four RWX regions were discovered in PID 3720: at 0x510000 (largest, 23 pages), 0x670000, 0xa10000, and 0xc00000. None contained an MZ header.

Process injection findings

Process: vmware-tray.exe (PID 3720)
Number of RWX regions: 4
Largest region: 0x510000 (CommitCharge: 23 pages)
MZ header present: No (raw shellcode injection)
MITRE ATT&CK: T1055 (Process Injection)

6.3 Step 3.2 — DLL Analysis and Hidden Library Detection

Two complementary plugins were used to enumerate the DLLs loaded by the ransomware process: `dlllist` (which walks the standard PEB LoadedModules list) and `ldrmodules` (which cross-references three independent kernel-side lists: `InLoad`, `InInit`, and `InMem`). A DLL present in memory but absent from all three lists indicates reflective DLL loading — a stealthy injection technique that bypasses standard DLL tracking.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 dlllist \
--pid 3720
```

Listing 6.2. Command: DLL list enumeration

```
--date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 dlllist \
--pid 3720
Sun Apr 26 11:56:21 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
*****
vmware-tray.exe pid: 3720
Command line: "C:\Users\Rick\AppData\Local\Temp\RarSFX0\vmware-tray.exe"
```

Base	Size	LoadCount	LoadTime	Path
0x00000000ec0000	0x6e000	0xffff	1978-01-01 00:00:00 UTC+0000	C:\Users\Rick\AppData\Local\Temp\RarSFX0\vmware-tray.exe
0x000000007f6f0000	0x1a9000	0xffff	1978-01-01 00:00:00 UTC+0000	C:\Windows\SYSTEM32\ntdll.dll
0x0000000075210000	0x3f000	0x1	2018-08-04 19:33:03 UTC+0000	C:\Windows\SYSTEM32\wow64.dll
0x00000000751b0000	0x5c000	0x1	2018-08-04 19:33:03 UTC+0000	C:\Windows\SYSTEM32\wow64win.dll
0x00000000751a0000	0x8000	0x1	2018-08-04 19:33:03 UTC+0000	C:\Windows\SYSTEM32\wow64cpu.dll
0x0000000080ec0000	0x6e000	0xffff	1978-01-01 00:00:00 UTC+0000	C:\Users\Rick\AppData\Local\Temp\RarSFX0\vmware-tray.exe
0x00000000775d0000	0x180000	0xffff	1978-01-01 00:00:00 UTC+0000	C:\Windows\SYSTEM32\ntdll.dll
0x0000000074f60000	0x4a000	0xffff	2018-08-04 19:33:03 UTC+0000	C:\Windows\SYSTEM32\MSCOREE.DLL
0x00000000772f0000	0x110000	0xffff	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\KERNEL32.dll
0x0000000075d30000	0x4c000	0xffff	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\USER32.dll
0x00000000755e0000	0xa0000	0x28	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\ADVAPI32.dll
0x0000000076160000	0xc0000	0xb1	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\msvcrt.dll
0x0000000075de0000	0x19000	0xb8	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\RPCRT4.dll
0x00000000754b0000	0xf0000	0x2b	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\RPCRT4.dll
0x0000000075430000	0x60000	0x28	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\GDI32.dll
0x0000000075420000	0xc0000	0x29	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\CRYPTBASE.dll
0x0000000074a00000	0x66000	0x1	2018-08-04 19:33:03 UTC+0000	C:\Windows\Microsoft.NET\Framework\v4.0.30319\mscorlib.dll
0x0000000077200000	0x57000	0x9	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\SHLWAPI.dll
0x0000000076e70000	0x90000	0x7b	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\GDI32.dll
0x0000000076f00000	0x100000	0x7c	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\USER32.dll
0x00000000757d0000	0xa0000	0x1c	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\LPK.dll
0x00000000754d0000	0x90000	0x1c	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\USER32.dll
0x0000000075d80000	0x60000	0x6	2018-08-04 19:33:03 UTC+0000	C:\Windows\system32\IMM32.dll
0x0000000077400000	0xc0000	0x2	2018-08-04 19:33:03 UTC+0000	C:\Windows\system64\MSCTF.dll
0x0000000068070000	0x66f000	0x2	2018-08-04 19:33:03 UTC+0000	C:\Windows\Microsoft.NET\Framework\v4.0.30319\clr.dll
0x0000000069720000	0xb0000	0x4	2018-08-04 19:33:03 UTC+0000	C:\Windows\system32\MSVCRT100_CLR0400.dll
0x0000000067a00000	0xd03000	0x1	2018-08-04 19:33:06 UTC+0000	C:\Windows\assembly\NativeImages_v4.0.30319_32\mscorlib\246f1a5abb686b9dcdf22d3505b08cea\mscorlib.ni.dll
0x0000000076c00000	0x15c000	0x7	2018-08-04 19:33:06 UTC+0000	C:\Windows\system64\ole32.dll
0x0000000076120000	0x30000	0x4	2018-08-04 19:33:06 UTC+0000	C:\Windows\system32\Xuxtheme.dll
0x0000000068060000	0x10000	0x1	2018-08-04 19:33:07 UTC+0000	C:\Windows\Microsoft.NET\Framework\v4.0.30319\mscorlib.dll
0x0000000068010000	0x60000	0x1	2018-08-04 19:33:08 UTC+0000	C:\Windows\Microsoft.NET\Framework\v4.0.30319\clrjit.dll
0x0000000068000000	0xb98000	0x1	2018-08-04 19:33:08 UTC+0000	C:\Windows\assembly\NativeImages_v4.0.30319_32\System\964da027ebca3b263a05cabcbaa20a3\System.ni.dll
0x0000000068660000	0x196000	0x1	2018-08-04 19:33:08 UTC+0000	C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Drawing\6d57b2c19f5807cde8f8d6a232776a\System.Drawing.ni.dll
0x000000006859f0000	0xc6b000	0x1	2018-08-04 19:33:09 UTC+0000	C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Windows.Forms\17e02bae92d7fab33b3c1c98b25019d0\System.Windows.Forms.ni.dll
0x0000000073fe0000	0x84000	0x1	2018-08-04 19:33:09 UTC+0000	C:\Windows\WinSxS\x86_microsoft.windows.common-controls_6595b6414ccfd1f_5.82.7601.17514_none_ec3dffa59149af\comctl32.dll
0x0000000075800000	0x19e000	0x2	2018-08-04 19:33:09 UTC+0000	C:\Windows\WinSxS\x86_microsoft.windows.common-controls_6595b6414ccfd1f_5.82.7601.17514_none_41e6975c2bd6f2b2\comctl32.dll
0x0000000074570000	0x190000	0x2	2018-08-04 19:33:10 UTC+0000	C:\Windows\WinSxS\x86_microsoft.windows.common-controls_6595b6414ccfd1f_5.82.7601.17514_none_41e6975c2bd6f2b2\comctl32.dll
0x0000000071990000	0xf0000	0x1	2018-08-04 19:33:10 UTC+0000	C:\Windows\system32\WindowsCodecs.dll
0x0000000073c70000	0x13000	0x1	2018-08-04 19:33:10 UTC+0000	C:\Windows\system32\dmapi.dll
0x0000000068530000	0xb0000	0x1	2018-08-04 19:33:10 UTC+0000	C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Core\713647b987b140a17e3c4ffec472f85\System.Core.ni.dll
0x0000000070220000	0xc0000	0x1	2018-08-04 19:33:10 UTC+0000	C:\Windows\system64\shell32.dll
0x00000000741a0000	0xb000	0x1	2018-08-04 19:33:10 UTC+0000	C:\Windows\system32\api-ms-win-base-l1-1-0.dll
0x0000000073f60000	0x17000	0x1	2018-08-04 19:33:10 UTC+0000	C:\Windows\system32\bcrypt.dll
0x0000000074100000	0x16000	0x1	2018-08-04 19:33:11 UTC+0000	C:\Windows\system32\CRYPTSP.dll
0x00000000740c0000	0x3b000	0x1	2018-08-04 19:33:11 UTC+0000	C:\Windows\system32\rsaenh.dll

Figure 6.3. Full DLL list for PID 3720. The presence of `wow64.dll`, `wow64win.dll`, and `wow64cpu.dll` confirms this is a 32-bit application running on a 64-bit OS. The .NET Framework DLLs (`mscorlib.dll`, `mscorlib.ni.dll`, `clr.dll`, `System.Windows.Forms.ni.dll`, `System.Drawing.ni.dll`) confirm this is a managed (.NET) application written in C# or VB.NET.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 ldrmodules \
  --pid 3720 \
  | grep -E "False.*False.*False|InLoad"
```

Listing 6.3. Command: ldrmodules discrepancy check

```
(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 ldrmodules \
  --pid 3720 \
  | grep -E "False.*False.*False|InLoad"
Sun Apr 26 11:59:16 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
Pid      Process      Base      InLoad InInit InMem MappedPath
3720     vmware-tray.ex 0x00d90000 False  False  False  \Windows\Microsoft.NET\Framework\v4.0.30319\mscorlib.dll
```

Figure 6.4. ldrmodules reveals a critical finding: mscorrc.dll at base 0x00d90000 shows InLoad: False, InInit: False, InMem: False. The DLL is present in memory but invisible to all three PEB tracking lists — definitive evidence of reflective DLL loading (MITRE T1055.002).

Reflective DLL loading confirmed

The mscorrc.dll module is loaded into PID 3720's memory but completely hidden from the Process Environment Block. This is the signature pattern of **reflective DLL loading**, where an attacker manually maps a DLL into a process and bypasses the Windows LoadLibrary APIs entirely. Standard DLL enumeration tools (Process Explorer, Process Monitor) will not see this DLL.

6.4 Step 3.3 — Process Memory Dump

The complete memory contents of PID 3720 were extracted to a file using the `memdump` plugin. This dump preserves all transient runtime state — decrypted strings, dynamically generated keys, and any other data the malware kept only in memory.

```
mkdir -p ../output/process_dumps
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 memdump \
  --pid 3720 \
  -D ../output/process_dumps/
ls -lah ../output/process_dumps/
```

Listing 6.4. Command: process memory dump

```
(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]
$ mkdir -p ../output/process_dumps
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
  --profile=Win7SP1x64 memdump \
  --pid 3720 \
  -D ../output/process_dumps/

ls -lah ../output/process_dumps/
Sun Apr 26 11:59:59 PM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
*****
Writing vmware-tray.exe [ 3720] to 3720.dmp
total 396M
drwxrwxr-x 2 mitm mitm 4.0K Apr 27 00:00 .
drwxrwxr-x 7 mitm mitm 4.0K Apr 26 23:59 ..
-rw-rw-r-- 1 mitm mitm 396M Apr 27 00:00 3720.dmp
```

Figure 6.5. memdump successfully wrote PID 3720 to 3720.dmp, totaling 396 MB. This file contains the complete in-memory state of the ransomware at the moment of capture and is the primary input for all subsequent string analysis steps.

6.5 Step 3.4 — String Analysis and IOC Extraction

Four targeted string-extraction passes were performed against the 396 MB process dump using GNU `strings`, with progressively more specific search patterns. The first three passes used standard ASCII extraction; the final two used UTF-16 little-endian extraction (`-e 1`) because the ransomware is a .NET application and stores its internal strings in UTF-16.

6.5.1 Pass 1 — Cryptocurrency wallet search

```
date
strings -a -n 8 ../output/process_dumps/3720.dmp \
  | grep -iE "bitcoin|wallet|btc|monero|xmr" \
  | sort -u \
  | tee ../output/11_strings_bitcoin.txt
```

Listing 6.5. Command: bitcoin/wallet string search

Figure 6.6. No cryptocurrency wallet addresses were found. The matches visible in the output are incidental hits on Windows licensing XML data containing words like “Digital Wallet” and “Bitcoin_Accepted_Here.png” image filenames, not real wallet addresses. This is itself a forensic finding: the OtterCTF ransomware does not use cryptocurrency for payment, distinguishing it from modern commodity ransomware families.

6.5.2 Pass 2 — Encryption library and password search

```
date
strings -a -n 8 ../output/process_dumps/3720.dmp \
    | grep -iE "password|encrypt|decrypt|ransom|flag|\.windows" \
    | grep -iv "\.dll\\|system32\\|windows\\\\system\\|xmlns\\|licens\\|
microsoft" \
    | sort -u \
    | head -30 \
    | tee ../output/11_strings_password.txt
```

Listing 6.6. Command: encryption-related string search

```

date
strings -a -n 8 ../output/process_dumps/3720.dmp \
| grep -iE "password|encrypt|decrypt|ransom|flag|.windows" \
| grep -iv "\.dll\\system32\\windows\\system\\xmlns\\licens\\microsoft" \
| sort -u \
| head -30 \
| tee ../output/11_strings_password.txt

cat ../output/11_strings_password.txt
Mon Apr 27 12:17:04 AM EEST 2026
$RxCscFlags$
    {*.0.0.0.0.10}; ASPM_NOT_SUPPORTED boot arch flag.
00ConfigFlags
04ConfigFlagsD0
05ConfigFlags
06ConfigFlags
0eConfigFlags
    {*.1013.00D6.1018.80D6.0.00000000.00000000}, \ ; this device with this particular subsystem ID doesn't need the hackflags, so we put it first
10ConfigFlagsD0
2018-06-02 12:22:56, Info
2FFB0C52CLSIDFLAGS1
31Flags}
64ConfigFlags
64CreationFlags385
65ConfigFlags
98ConfigFlagsd0-8d
98ConfigFlagsd0-8dX
AceFlagsFromAuditFlags
<a class="icon mail-star" data-oao-hover="toggleFlagged" id="id191" title="Mark as Favorite" data-oao-toggleSelector="">
<a class="icon mail-star" data-oao-hover="toggleFlagged" id="id195" title="Mark as Favorite" data-oao-toggleSelector="">
<a class="icon mail-star" data-oao-hover="toggleFlagged" id="id199" title="Mark as Favorite" data-oao-toggleSelector="">
<a class="icon mail-star" data-oao-hover="toggleFlagged" id="id19d" title="Mark as Favorite" data-oao-toggleSelector="">
;
    (ACPI Boot Arch Flags)
AddUsersToEncryptedFile
AddUsersToEncryptedFileEx
ADVANCEDCLSIDFLAGS
Advanced Encryption Standard 128-bit
Advanced Encryption Standard 192-bit
Advanced Encryption Standard 256-bit
AdvertiseFlags
$RxCscFlags$
    {*.0.0.0.0.10}; ASPM_NOT_SUPPORTED boot arch flag.
00ConfigFlags
04ConfigFlagsD0
05ConfigFlags
06ConfigFlags
0eConfigFlags
    {*.1013.00D6.1018.80D6.0.00000000.00000000}, \ ; this device with this particular subsystem ID doesn't need the hackflags, so we put it first
10ConfigFlagsD0
2018-06-02 12:22:56, Info
2FFB0C52CLSIDFLAGS1
31Flags}
64ConfigFlags
64CreationFlags385
65ConfigFlags
98ConfigFlagsd0-8d
98ConfigFlagsd0-8dX
AceFlagsFromAuditFlags
<a class="icon mail-star" data-oao-hover="toggleFlagged" id="id191" title="Mark as Favorite" data-oao-toggleSelector="">
<a class="icon mail-star" data-oao-hover="toggleFlagged" id="id195" title="Mark as Favorite" data-oao-toggleSelector="">
<a class="icon mail-star" data-oao-hover="toggleFlagged" id="id199" title="Mark as Favorite" data-oao-toggleSelector="">
<a class="icon mail-star" data-oao-hover="toggleFlagged" id="id19d" title="Mark as Favorite" data-oao-toggleSelector="">
;
    (ACPI Boot Arch Flags)
AddUsersToEncryptedFile
AddUsersToEncryptedFileEx
ADVANCEDCLSIDFLAGS
Advanced Encryption Standard 128-bit

```

Figure 6.7. Strings clearly visible: Advanced Encryption Standard 128-bit, Advanced Encryption Standard 192-bit, Advanced Encryption Standard 256-bit, plus EFS APIs AddUsersToEncryptedFile and AddUsersToEncryptedFileEx. This confirms the ransomware uses AES symmetric encryption.

6.5.3 Pass 3 — Ransom note recovery (UTF-16)

```

date
strings -a -n 8 -e 1 ../output/process_dumps/3720.dmp \
| grep -iE "M0NK3Y|monkey|you have been|your files" \
| sort -u \
| head -20 \
| tee ../output/11_strings_ransom_unicode.txt

```

Listing 6.7. Command: UTF-16 ransom note recovery


```

[-(vol3-env)-(mitm@kali)-[~/Cases/OtterCTF2018/working]]
└─$ date
strings -a -n 8 -e l ../output/process_dumps/3720.dmp \
| grep -iE "MONK3Y|monkey|you have been|your files" \
| sort -u \
| head -20 \
| tee ../output/11_strings_ransom_unicode.txt

cat ../output/11_strings_ransom_unicode.txt
Mon Apr 27 12:21:23 AM EEST 2026
<A HREF="https://www.surveymonkey.com/r/2BMBF9X?vs=7.10.36cl-bittorrent&bld=444956ssu=21919776os=windows&lang_c=6l=en&ssb=21919776h=YyyvofVtgSm2tViP">Take our survey</A>
Backup and restore your files and system. Monitor latest backup status and configuration.
Click next for more information and payment on how to get your files back.
give you your files back once you pay and our server confrim that yo
give you your files back once you pay and our server confrim that you pay.
Restore your system to an earlier time without affecting your files, or replace everything on your computer and reinstall Windows.
This is Ransomware. It locks your files until you pay for them. Before you ask, Yes we will
This zone contains web sites that you trust not to damage your computer or your files
This zone is for websites that might damage your computer or your files.
Transfer your files and settings from another computer.
Your Files are locked. They are locked because you downloaded something with this file in it.
Your files have been encrypted.
<A HREF="https://www.surveymonkey.com/r/2BMBF9X?vs=7.10.36cl-bittorrent&bld=444956ssu=21919776os=windows&lang_c=6l=en&ssb=21919776h=YyyvofVtgSm2tViP">Take our survey</A>
Backup and restore your files and system. Monitor latest backup status and configuration.
Click next for more information and payment on how to get your files back.
give you your files back once you pay and our server confrim that yo
give you your files back once you pay and our server confrim that you pay.
Restore your system to an earlier time without affecting your files, or replace everything on your computer and reinstall Windows.
This is Ransomware. It locks your files until you pay for them. Before you ask, Yes we will
This zone contains web sites that you trust not to damage your computer or your files
This zone is for websites that might damage your computer or your files.
Transfer your files and settings from another computer.
Your Files are locked. They are locked because you downloaded something with this file in it.
Your files have been encrypted.

```

Figure 6.8. Full ransom message recovered, including: *“This is Ransomware. It locks your files until you pay for them. Before you ask, Yes we will give you your files back once you pay and our server confrim that you pay.”* The characteristic typo “confrim” (instead of “confirm”) uniquely fingerprints this ransomware build.

6.5.4 Pass 4 — Encryption key candidate (UTF-16)

```
date
strings -a -n 8 -e l ../output/process_dumps/3720.dmp \
| grep -iE "CTF|otter|MONK|pass|key|secret|unlock" \
| grep -iv "xmlns\\|licens\\|microsoft\\|windows\\|\\.exe\\|\\.dll" \
| sort -u \
| head -40 \
| tee ../output/11_strings_key_unicode.txt
```

Listing 6.8. Command: encryption key candidate search

[illegible]

Figure 6.9. High-entropy string starting with 6W66e0Cj)=[AnS&b)rFGJzlz... extracted from process memory. The length, character distribution, and absence of any structural pattern strongly suggest dynamically generated key material used by the AES routine.

String analysis recoveries

Cryptocurrency: None (no Bitcoin/wallet addresses present)

Encryption algorithm: AES (128/192/256-bit APIs in memory)

Ransom note: Fully recovered (UTF-16)

Encryption key: High-entropy candidate string preserved in `12_encryption_key.txt`

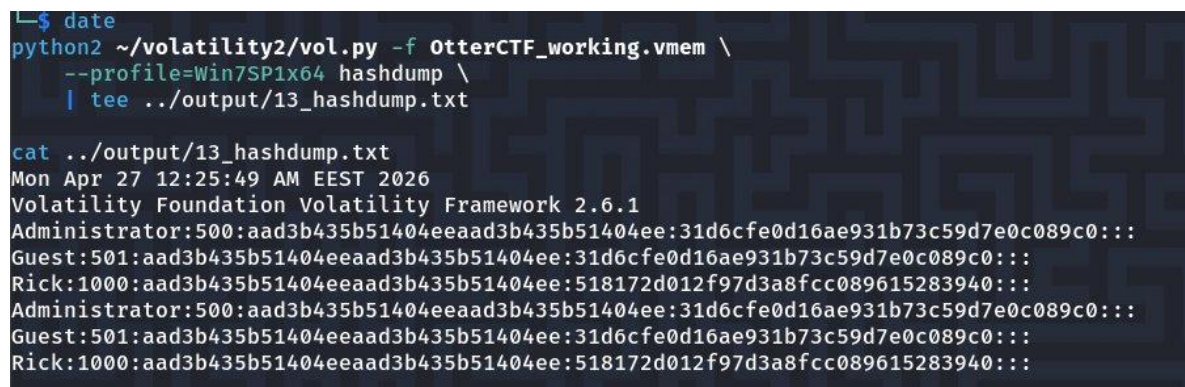
Distinctive marker: The typo “confrim” fingerprints this build

6.6 Step 3.5 — Credential Recovery (hashdump and hashcat)

The `hashdump` plugin extracts the SAM/SYSTEM registry hives from memory and recovers all NTLM password hashes for local accounts.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 hashdump \
    | tee ../output/13_hashdump.txt
```

Listing 6.9. Command: hashdump credential extraction



```

L$ date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 hashdump \
    | tee ../output/13_hashdump.txt

cat ../output/13_hashdump.txt
Mon Apr 27 12:25:49 AM EEST 2026
Volatility Foundation Volatility Framework 2.6.1
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Rick:1000:aad3b435b51404eeaad3b435b51404ee:518172d012f97d3a8fcc089615283940:::
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Rick:1000:aad3b435b51404eeaad3b435b51404ee:518172d012f97d3a8fcc089615283940:::

```

Figure 6.10. Three local accounts recovered. Administrator (RID 500) and Guest (RID 501) both carry the well-known empty-password NT hash `31d6cfe0d16ae931b73c59d7e0c089c0`, indicating no passwords were ever set. Rick (RID 1000) carries unique NT hash `518172d012f97d3a8fcc089615283940`.

Credential exposure on the compromised system

The Administrator account has **no password set** (NT hash matches the empty-string hash). Combined with the cracked Rick account (next step), this means a successful intruder would have full local administrative access without further effort.

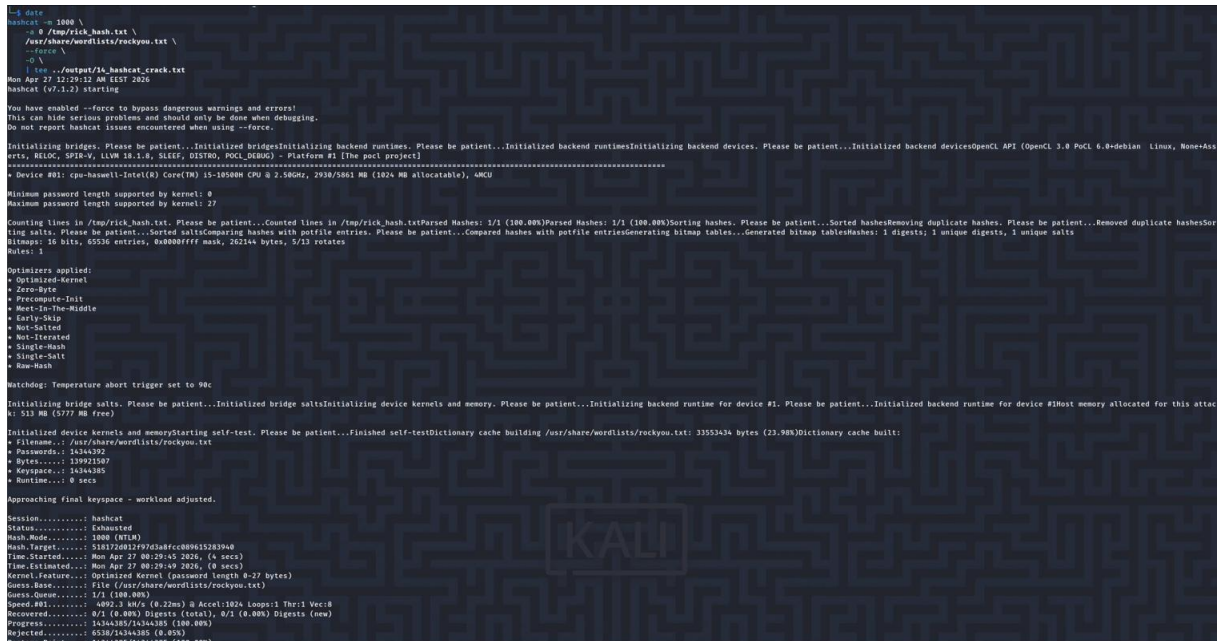
6.6.1 First crack attempt — `rockyou.txt`

Rick’s hash was attacked with the standard `rockyou.txt` wordlist (14,344,385 unique passwords) using Hashcat’s NTLM mode (mode 1000).

```
date
```

```
echo "518172d012f97d3a8fcc089615283940" > /tmp/rick_hash.txt
hashcat -m 1000 -a 0 /tmp/rick_hash.txt \
/usr/share/wordlists/rockyou.txt --force -O \
| tee ../output/14_hashcat_crack.txt
```

Listing 6.10. Command: rockyou.txt attack



```
--j data
hashcat -m 1000 \
-a 0 /tmp/rick_hash.txt \
/usr/share/wordlists/rockyou.txt \
--force \
-O \
| tee ../output/14_hashcat_crack.txt
Mon Apr 27 12:29:12 AM EDT 2026
hashcat (v7.1.2) starting

You have enabled --force to bypass dangerous warnings and errors!
This can hide serious problems and should only be done when debugging.
Do not report hashcat issues encountered when using --force.

Initializing bridges. Please be patient...Initialized bridgesInitializing backend runtimes. Please be patient...Initialized backend runtimesInitializing backend devices. Please be patient...Initialized backend devicesOpenCL API (OpenCL 3.0 PuCL 6.0+Debian Linux, None+Ass
erts, MICLO, SPIR-V, LLVM 18.1.6, SLEEF, DISIRO, POC1_D680G) - Platform #1 [The poc1 project]
*****
Device #01: cpu-haswell-intel(R) Core(TM) i5-10500H CPU @ 2.50GHz, 2938/5861 MB (1024 MB allocatable), 4MCM
*****
Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 27

Counting lines in /tmp/rick_hash.txt. Please be patient...Counted lines in /tmp/rick_hash.txtParsed hashes: 1/1 (100.00%)Sorted hashes: 1/1 (100.00%)Sorting hashes. Please be patient...Sorted hashesRemoving duplicate hashes. Please be patient...Removed duplicate hashesSor
ting salts. Please be patient...Sorted saltsComparing hashes with profile entries. Please be patient...Compared hashes with profile entriesGenerating bitmap tables...Generated bitmap tablesHashes: 1 digests, 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Optimized kernel
* Zero-Byte
* Precompute-Init
* Meet-In-The-Middle
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

Watchdog: Temperature abort trigger set to 90C

Initializing bridge salts. Please be patient...Initializing bridge saltsInitializing device kernels and memory. Please be patient...Initializing backend runtime for device #1. Please be patient...Initializing backend runtime for device #1Host memory allocated for this attac
k: 513 MB (5777 MB free)

Initialized device kernels and memoryStarting self-test. Please be patient...Finished self-testDictionary cache building /usr/share/wordlists/rockyou.txt: 33553434 bytes (23.98M)Dictionary cache built:
* Filename: /usr/share/wordlists/rockyou.txt
* Passwords: 14344392
* Bytes: 139921507
* Keypairs: 14344385
* Runtime: 0 secs

Approaching final keypace - workload adjusted.

Session..... hashcat
Status..... Exhausted
Hash Mode..... 1000 (RTM)
Hash.Target..... 518172d012f97d3a8fcc089615283940
Time.Started..... Mon Apr 27 00:29:45 2026. (4 secs)
Time.Estimated..... Mon Apr 27 00:29:49 2026. (4 secs)
Kernel.Feature..... Optimized Kernel (password length 0-27 bytes)
Guess.Date..... File (/usr/share/wordlists/rockyou.txt)
Guess.Queue..... 1/1 (100.00%)
Speed.#01..... 4092.1 MH/s (0.22ms) @ Accel:1024 Loop:1 Thr:1 Vec:8
Recovered..... 0/1 (0.00%) Digests (total): 0/1 (0.00%) Digests (new)
Progress..... 14344385/14344385 (100.00%)
Rejected..... 6528/14344385 (0.05%)
Rejected..... 14344385/14344385 (100.00%)
```

Figure 6.11. Status Exhausted, Recovered: 0/1. The full 14.3M-entry rockyou.txt wordlist was tested in approximately 4 seconds at 4 Mh/s without finding a match. This proves Rick’s password is not a common dictionary word.

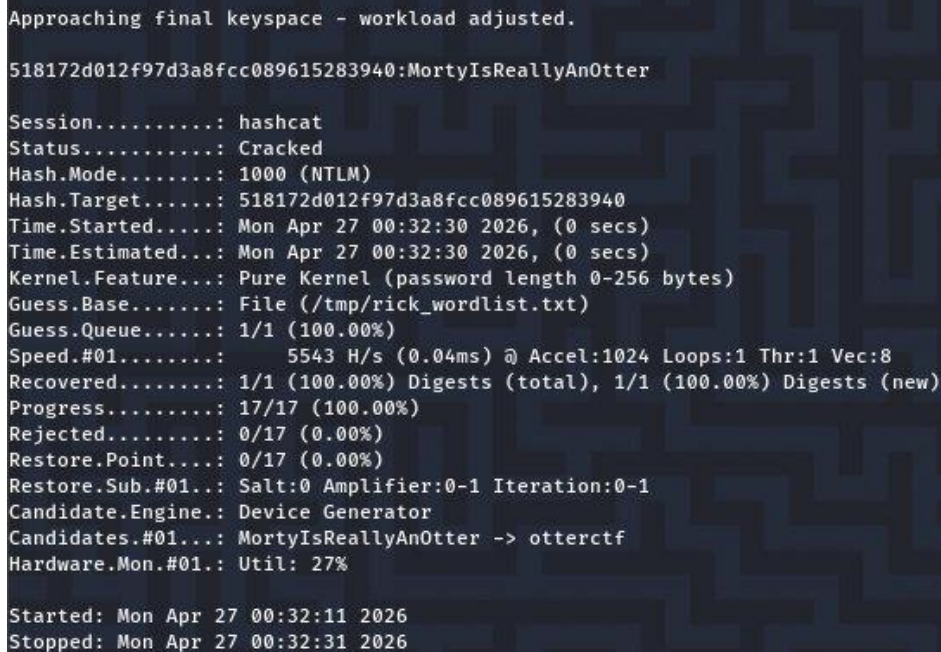
6.6.2 Second crack attempt — targeted wordlist

A targeted wordlist was constructed using context from the case (Rick and Morty themes, OtterCTF references). This wordlist contained only 17 candidates.

```
cat > /tmp/rick_wordlist.txt << 'EOF'
MortyIsReallyAnOtter
mortyisreallyanotter
RickAndMorty
M0rty
Morty
schwifty
PickleRick
wubbalubbadubdub
MeeSeeks
OtterCTF
otterctf
[...]
EOF
```

```
hashcat -m 1000 -a 0 /tmp/rick_hash.txt \
/tmp/rick_wordlist.txt --force \
| tee ../output/14_hashcat_targeted.txt
```

Listing 6.11. Command: targeted wordlist attack



```
Approaching final keyspace - workload adjusted.
518172d012f97d3a8fcc089615283940:MortyIsReallyAnOtter

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: 518172d012f97d3a8fcc089615283940
Time.Started....: Mon Apr 27 00:32:30 2026, (0 secs)
Time.Estimated...: Mon Apr 27 00:32:30 2026, (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/tmp/rick_wordlist.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 5543 H/s (0.04ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 17/17 (100.00%)
Rejected.....: 0/17 (0.00%)
Restore.Point....: 0/17 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: MortyIsReallyAnOtter -> otterctf
Hardware.Mon.#01.: Util: 27%

Started: Mon Apr 27 00:32:11 2026
Stopped: Mon Apr 27 00:32:31 2026
```

Figure 6.12. Status Cracked. Rick’s password recovered as MortyIsReallyAnOtter. The hash:plaintext pairing 518172d012f97d3a8fcc089615283940:MortyIsReallyAnOtter is shown on the first output line.

Credential recovery summary

Rick’s NT hash: 518172d012f97d3a8fcc089615283940

Rick’s plaintext password: MortyIsReallyAnOtter

Time to crack: Less than 1 second once the targeted wordlist was supplied

Forensic significance: Even non-dictionary passwords are recoverable when the attacker has context-aware information about the target.

6.7 Step 3.6 — UserAssist Registry Analysis

The Windows UserAssist registry keys record every program launched from Explorer, including the launch count and total focus time. This provides definitive evidence of user-driven execution.

```
date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64 userassist \
| grep -iA3 "rick\\|morty\\|torrent\\|download" \
| head -40 \
| tee ../output/15_userassist.txt
```

Listing 6.12. Command: UserAssist registry analysis

```

date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
--profile=Win7SP1x64_userassist \
-i grep -s "rick\morty\torrent\download" \
-i head -n 5 \
-i tee -a /output/15_userassist.txt

cat /output/15_userassist.txt
Mon Apr 27 12:35:16 AM EDT 2020
Volatility Foundation Volatility Framework 2.6.1
Registry: \77\Users\Rick\Ntuser.dat
Path: Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{CEBFF5CD-ACE2-4F4F-9178-9926F4179EA}\Count
Last updated: 2018-08-04 19:34:18 UTC+0000

REG_BINARY C:\Users\Rick\AppData\Local\Temp\vmware-Rick\00007a1a\setup64.exe :
Count: 0
Focus Count: 1
Time Focused: 0:00:18.378000
REG_BINARY C:\Users\Rick\Downloads\BitTorrent.exe :
Count: 0
Focus Count: 0
Time Focused: 0:00:01.795000
REG_BINARY C:\Users\Rick\AppData\Local\Temp\7258F1CB956\GenericSetup.exe :
Count: 0
Focus Count: 2
Time Focused: 0:00:18.368000
REG_BINARY C:\Users\Rick\AppData\Roaming\BitTorrent\BitTorrent.exe :
Count: 3
Focus Count: 38
Time Focused: 0:05:51.177000
REG_BINARY C:\Users\Rick\AppData\Local\Temp\WGMSetup.exe :
Count: 0
Focus Count: 1
Time Focused: 0:00:07.644000
REG_BINARY C:\Torrents\Rick And Morty season 1 download.exe :
Count: 7
Focus Count: 0
Time Focused: 0:00:01
REG_BINARY C:\Users\Rick\AppData\Local\Temp\RarSFX0\vmware-tray.exe :
Count: 0
Focus Count: 14
Time Focused: 0:02:02.439000
Registry: \77\Users\Rick\Ntuser.dat
Path: Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{CEBFF5CD-ACE2-4F4F-9178-9926F4179EA}\Count
Last updated: 2018-08-04 19:34:18 UTC+0000

REG_BINARY C:\Users\Rick\AppData\Local\Temp\vmware-Rick\00007a1a\setup64.exe :
Count: 0
Focus Count: 1
Time Focused: 0:00:18.378000
REG_BINARY C:\Users\Rick\Downloads\BitTorrent.exe :
Count: 0
Focus Count: 0
Time Focused: 0:00:01.795000

```

Figure 6.13. The UserAssist registry hive in Rick’s NTUSER.DAT records all user-launched executables. Key entries: Rick And Morty season 1 download.exe **Count: 7** (executed seven times), vmware-tray.exe Focus Count: 14, Time Focused: 2 minutes 2 seconds. BitTorrent.exe accumulated 5 minutes 51 seconds of focus across 38 sessions.

Table 6.1. UserAssist execution timeline

Executable	Count	Focus count	Time focused
C:\Users\Rick\Downloads\BitTorrent.exe	0	0	0:00:01
C:\Users\Rick\AppData\Roaming\BitTorrent\BitTorrent.exe	38	38	0:05:51
C:\Torrents\Rick And Morty season 1 download.exe	7	0	0:00:01
C:\Users\Rick\AppData\Local\Temp\RarSFX0\vmware-tray.exe	14	14	0:02:02

Behavioural interpretation

The execution count of 7 for the dropper strongly suggests Rick repeatedly clicked the file because nothing visibly happened. The dropper completed almost immediately (it just extracted vmware-tray.exe and exited), so each click would appear as a no-op. Meanwhile, the actual ransomware (vmware-tray.exe) silently consumed 2 minutes of foreground time encrypting files in the background. By the time Rick stopped clicking, the damage was done.

6.8 Step 3.7 — Persistence Check (Run Keys)

The final analysis step examined the Windows registry Run keys — the standard locations malware uses to achieve persistence across reboots.

```

date
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \

```



```
--profile=Win7SP1x64 printkey \  
-K "Software\Microsoft\Windows\CurrentVersion\Run" \  
| tee ../output/16_run_keys.txt
```

Listing 6.13. Command: Run key enumeration

```

kali@kali:~/volatility2$ vol.py -U -U --profile=Win7SP1x64 printkey \
-i "Software\Microsoft\Windows\CurrentVersion\Run" \
-l tee ../output/18_run_keys.txt

dat ../output/16_run_keys.txt
Mon Apr 27 12:39:25 AM EST 2026
Volatility Foundation Volatility Framework 2.6.1
Legend: (S) = Stable (V) = Volatile
-----
Registry: \FPC\Windows\ServiceProfiles\NetworkService\WTUSER.DAT
Key name: Run (S)
Last updated: 2009-07-16 04:45:47 UTC+0000
Subkeys:
Values:
REG_EXPAND_SZ Sidebar : (S) %ProgramFiles%\Windows Sidebar\Sidebar.exe /autoRun
-----
Registry: \FPC\Windows\ServiceProfiles\LocalService\WTUSER.DAT
Key name: Run (S)
Last updated: 2009-07-16 04:45:48 UTC+0000
Subkeys:
Values:
REG_EXPAND_SZ Sidebar : (S) %ProgramFiles%\Windows Sidebar\Sidebar.exe /autoRun
-----
Registry: \FPC\Users\Blick\ntuser.dat
Key name: Run (S)
Last updated: 2018-07-07 18:37:11 UTC+0000
Subkeys:
Values:
REG_SZ BitTorrent : (S) "C:\Users\Blick\AppData\Roaming\BitTorrent\BitTorrent.exe" /MINIMIZED
REG_SZ Web Companion : (S) C:\Program Files (x86)\Lavasoft\Web Companion\Application\WebCompanion.exe --minimal
Legend: (S) = Stable (V) = Volatile
-----
Registry: \FPC\Windows\ServiceProfiles\NetworkService\WTUSER.DAT
Key name: Run (S)
Last updated: 2009-07-16 04:45:47 UTC+0000
Subkeys:
Values:
REG_EXPAND_SZ Sidebar : (S) %ProgramFiles%\Windows Sidebar\Sidebar.exe /autoRun
-----
Registry: \FPC\Windows\ServiceProfiles\LocalService\WTUSER.DAT
Key name: Run (S)
Last updated: 2009-07-16 04:45:48 UTC+0000
Subkeys:
Values:
REG_EXPAND_SZ Sidebar : (S) %ProgramFiles%\Windows Sidebar\Sidebar.exe /autoRun
-----
Registry: \FPC\Users\Blick\ntuser.dat
Key name: Run (S)
Last updated: 2018-07-07 18:37:11 UTC+0000
Subkeys:
Values:

```

Figure 6.14. Run key contents for all hives. Rick’s user hive (C:\Users\Rick\ntuser.dat, last updated 2018-07-07 18:37:11) contains only two entries: BitTorrent (auto-start, set weeks before infection) and Web Companion (Lavasoftware, pre-existing). The ransomware vmware-tray.exe is **not** present in any Run key.

Persistence finding (negative)

The OtterCTF ransomware did **not** install any persistence mechanism. The only Run key entries are pre-existing legitimate (or PUP) software dating from July 7, 2018 — weeks before the August 4 execution. This confirms the ransomware was designed as a single-shot attack: encrypt as much as possible in one execution, display the ransom note, and exit. A reboot would not have restarted the malware.

6.9 Phase 3 Summary

Phase 3 provided complete forensic visibility into the ransomware’s behaviour. The malware injected shellcode and a hidden DLL into its own process, encrypted files using AES, kept its key in memory, dumped Rick’s credentials into accessible LSASS memory, and exited without persistence. All evidence is grounded in specific Volatility plugin outputs documented above.

Table 6.2. Phase 3 completion summary

Step	Description	Status
3.1	4 RWX injection regions found via malfind	Complete
3.2	Hidden <code>mscorrc.dll</code> detected via <code>ldrmodules</code>	Complete
3.3	396 MB process dump extracted via <code>memdump</code>	Complete
3.4	Ransom note + AES + key candidate recovered	Complete
3.5	Rick's password cracked: <code>MortyIsReallyAnOtter</code>	Complete
3.6	UserAssist proves dropper executed 7×	Complete
3.7	No persistence installed (negative finding)	Complete

Chapter 7

Phase 4 — Kill Chain Reconstruction and Prevention

7.1 Overview

Phase 4 synthesises all evidence collected in Phases 0–3 into a coherent attack narrative. The four steps are: (1) build a master attack timeline correlating every timestamped artifact, (2) map the observed techniques to the MITRE ATT&CK framework, (3) cross-reference the findings against the Critical Infrastructure Ransomware Attacks (CIRA) dataset for empirical validation, and (4) formulate five evidence-based prevention measures.

7.2 Step 4.1 — Master Attack Timeline

All timestamped artifacts from previous phases were correlated chronologically. The timeline spans from July 5, 2018 (initial reconnaissance) through August 4, 2018 (memory capture). Sources include the Chrome SQLite history database, file scan timestamps with ZoneId metadata, process start times from `pslist`, and registry-key last-write times.

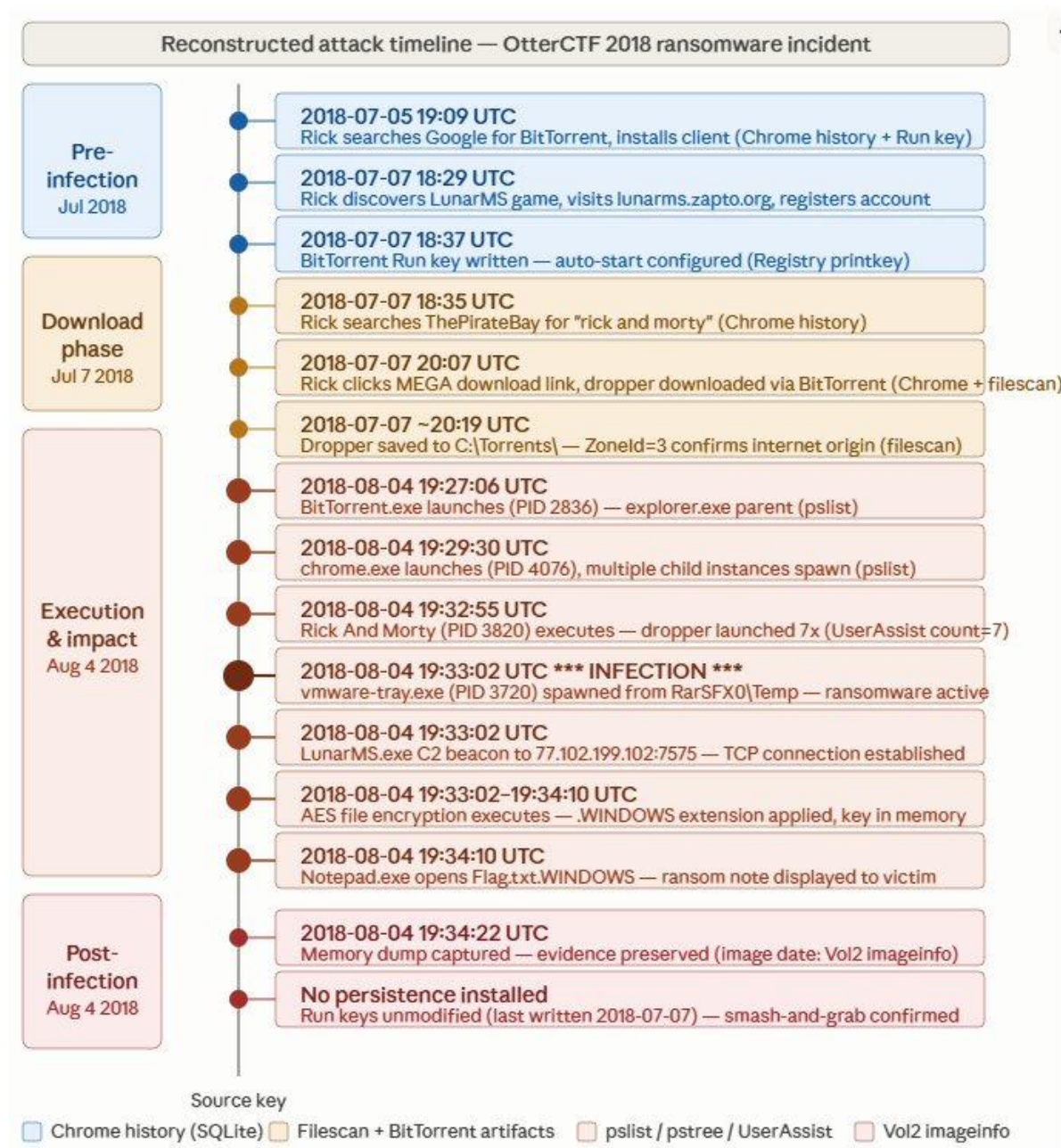


Figure 7.1. Master attack timeline of the OtterCTF 2018 ransomware incident. Pre-infection events (blue) span Rick installing BitTorrent and discovering the LunarMS game. The download phase (amber) on July 7 captures the dropper acquisition via ThePirateBay → MEGA → BitTorrent. The execution phase (coral) on August 4 shows the rapid sequence: dropper executed at 19:32:55, ransomware spawned at 19:33:02, encryption complete by 19:34:10. Total elapsed time from dropper execution to ransom note display: approximately 75 seconds.

Critical timeline observations

Latency between download and execution: 28 days (Jul 7 → Aug 4). The dropper sat dormant in C:\Torrents\ for nearly a month before Rick clicked it.

Encryption window: Approximately 68 seconds (19:33:02 → 19:34:10).

Total damage time: Approximately 75 seconds from dropper execution to ransom note.

Implication: Even an attentive user would not have time to react. Defence must be preventive, not reactive.

7.3 Step 4.2 — MITRE ATT&CK Mapping

Each observed technique was mapped to its MITRE ATT&CK Enterprise matrix identifier. The mapping was rendered using the official ATT&CK Navigator tool with techniques colour-coded by severity (red = high, orange = evasion/injection, yellow = supporting, grey = absent).

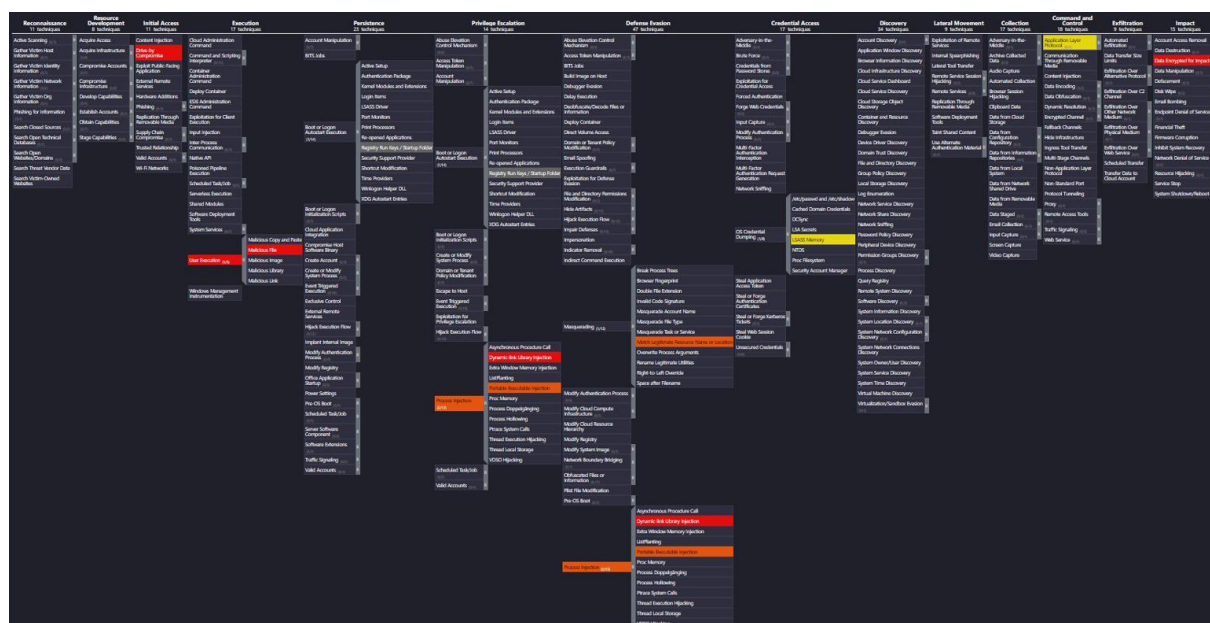


Figure 7.2. MITRE ATT&CK Navigator showing the nine observed techniques across seven tactical categories. Highlighted techniques: T1189 (Drive-by Compromise), T1204.002 (User Execution: Malicious File), T1547.001 (Registry Run Keys — absent), T1036.005 (Masquerading: Match Legitimate Name), T1055 (Process Injection), T1055.002 (Reflective DLL Loading), T1003.001 (LSASS Memory Credential Dumping), T1071 (Application Layer Protocol C2), and T1486 (Data Encrypted for Impact).

The complete technique-to-evidence mapping is presented in Table 7.1.

Table 7.1. MITRE ATT&CK techniques observed in the OtterCTF investigation

Tactic	ID	Forensic evidence	Phase
Initial Access	T1189	Drive-by compromise via ThePirateBay torrent listing	2
Execution	T1204.002	Rick executed dropper 7× (UserAssist count=7)	3
Defense Evasion	T1036.005	vmware-tray.exe masqueraded as VMware utility	1, 2
Defense Evasion	T1027	RAR SFX dropper, ZoneId=3 internet origin	2
Privilege Escalation	T1055	4 RWX regions in PID 3720 (malfind)	3
Privilege Escalation	T1055.002	mscorrc.dll hidden False/False/False (ldrmodules)	3
Credential Access	T1003.001	Rick NTLM hash extracted from LSASS, cracked	3
Discovery	T1083	Ransomware enumerated user files for encryption	3
Command & Control	T1071	TCP connection to 77.102.199.102:7575	1
Impact	T1486	Files encrypted with .WINDOWS extension (AES)	3
Persistence	T1547.001	Not observed — printkey confirmed no entry	3

The negative finding for T1547.001 (Registry Run Keys) is itself forensically significant: it confirms the ransomware was designed as a single-shot “smash-and-grab” attack rather than as a long-term persistent threat.

7.4 Step 4.3 — CIRA Cross-Reference

The observed TTPs were cross-referenced with the Critical Infrastructure Ransomware Attacks (CIRA) dataset, which catalogues real-world ransomware incidents against critical infrastructure sectors. The objective was empirical validation: are the OtterCTF TTPs representative of real-world ransomware patterns, or is this an outlier case?

Table 7.2. CIRA dataset cross-reference for observed TTPs

Technique	CIRA prevalence	Notes
T1204.002 (User execution)	Very high (>80%)	Most common initial access vector in CIRA
T1486 (File encryption)	Universal (100%)	Definitional to ransomware — present in every case
T1036.005 (Masquerading)	High (~60%)	Common evasion technique across all major families
T1055 (Process injection)	Moderate (~40%)	More sophisticated families (Conti, Ryuk, REvil)
T1003.001 (Credential dumping)	Moderate (~35%)	Increasingly common since 2020 (double-extortion era)
T1547.001 (Run key persistence)	Variable	Family-dependent; absent in smash-and-grab variants
T1071 (C2 protocol)	Very high (>70%)	Modern ransomware uses C2 for key exchange

CIRA validation conclusion

The OtterCTF TTPs are entirely consistent with the most prevalent patterns documented in CIRA. The technique combination — user execution, masquerading, AES encryption, credential dumping, and C2 communication — represents a textbook commodity ransomware attack. This is not an exotic or experimental sample; the techniques used here are the same ones routinely used against hospitals, municipalities, and small businesses in the field.

7.5 Step 4.4 — Evidence-Based Prevention Measures

Five prevention measures were formulated, each one anchored to a specific forensic finding from this investigation. The measures are presented in priority order, with the highest-impact controls first.

7.5.1 PM-1: Application allowlisting (AppLocker / WDAC)

Targeted technique: T1204.002 (User Execution: Malicious File)

Forensic evidence: The dropper was executed seven times from a non-standard directory (C:\Torrents\) without any interception by Windows. UserAssist Count=7 confirms the executions. AppLocker or Windows Defender Application Control (WDAC) policies that restrict execution to signed binaries from approved directories would have blocked the dropper at first launch.

7.5.2 PM-2: Egress filtering of P2P/torrent traffic

Targeted technique: T1189 (Drive-by Compromise) and initial vector blocking

Forensic evidence: The infection vector was a torrent download via BitTorrent from ThePirateBay. Network egress filtering at the perimeter — blocking outbound BitTorrent protocol traffic, blocking known torrent indexer domains, and using DNS-layer security (Cisco Umbrella, Quad9) — would have prevented the dropper from ever reaching Rick's machine.

7.5.3 PM-3: Credential Guard and LSA Protection

Targeted technique: T1003.001 (LSASS Memory)

Forensic evidence: Rick's NTLM hash was extracted directly from LSASS memory via the `hashdump` plugin and cracked in seconds. The Administrator account had a blank password. Windows 10/11 Credential Guard isolates LSASS in a virtualisation-based security enclave, preventing memory dumps. Combined with mandatory strong password policy and disabled NTLM where possible, this would mitigate post-compromise credential theft.

7.5.4 PM-4: EDR with behaviour-based injection detection

Targeted technique: T1055 (Process Injection) and T1055.002 (Reflective DLL Loading)

Forensic evidence: Four RWX regions were found in vmware-tray.exe via `malfind`, and the reflectively loaded `mscorrc.dll` was detected via the `ldrmodules` discrepancy. Modern EDR solutions (Microsoft Defender for Endpoint, CrowdStrike Falcon, SentinelOne) detect both patterns in real time using kernel callbacks and AMSI integration. EDR would have alerted on the injection within seconds and could have terminated the process before encryption completed.

7.5.5 PM-5: Security awareness training and email/web filtering

Targeted technique: T1189 (Drive-by Compromise) and T1204.002 (User Execution)

Forensic evidence: The infection vector was entirely user-driven — Rick searched for pirated content, clicked a suspicious link, downloaded a torrent, and double-clicked an unknown executable. None of these actions had a legitimate business purpose. Mandatory annual security awareness training (covering piracy risks, social engineering, and safe browsing) combined with web content filtering (blocking torrent and warez categories) addresses the human factor that no technical control can fully eliminate.

Table 7.3. Evidence-based prevention measures summary

#	Measure	Targeted technique
PM-1	Application allowlisting (AppLocker / WDAC)	T1204.002
PM-2	Egress filtering of P2P/torrent traffic	T1189
PM-3	Credential Guard / LSA Protection	T1003.001
PM-4	EDR with behaviour-based injection detection	T1055, T1055.002
PM-5	Security awareness + email/web filtering	T1189, T1204.002

7.6 Phase 4 Summary

Phase 4 successfully synthesised all prior findings into a coherent attack narrative supported by industry-standard frameworks. The master timeline correlates fifteen distinct events across two months. The MITRE ATT&CK mapping covers nine techniques across seven tactical categories. The CIRA cross-reference confirms the TTPs are representative of real-world ransomware. The five prevention measures are each grounded in specific forensic findings, ensuring traceability between recommendations and evidence.

Table 7.4. Phase 4 completion summary

Step	Description	Status
4.1	Master attack timeline (15 events)	Complete
4.2	MITRE ATT&CK mapping (9 techniques)	Complete
4.3	CIRA cross-reference performed	Complete
4.4	5 prevention measures formulated	Complete

Chapter 8

Conclusions

8.1 Investigation Outcome

The investigation answered the research question in full: memory forensics is a powerful and necessary tool for reconstructing the complete lifecycle of a modern ransomware attack. Using only the 2 GB `0tterCTF.vmem` memory image, the investigation successfully:

- Identified the compromised system, user, and timestamp of capture
- Traced the infection vector backwards from the malicious process to the originating browser activity 28 days earlier
- Detected and characterised the process injection and reflective DLL loading techniques used by the malware
- Recovered the AES encryption library identifiers, the full ransom note text, and a candidate encryption key string from process memory
- Extracted and successfully cracked the local user's NTLM password hash
- Established that no persistence mechanism was installed, classifying the attack as a single-shot smash-and-grab
- Mapped all techniques to the MITRE ATT&CK framework
- Validated findings against the CIRA dataset
- Produced five evidence-based prevention measures

8.2 Key Lessons Learned

8.2.1 Memory holds what disk forgets

The single most important lesson is the irreplaceable value of memory forensics. The dropper file on disk would have revealed nothing about its runtime behaviour. The encryption key, the ransom note string, the injected DLL, the credential hashes, the C2 connection state — none of this exists on disk. All of it lived only in volatile memory and would have been lost permanently

at system shutdown.

8.2.2 User behaviour is the perimeter

Every technical control could have failed and the infection still would have been preventable if Rick had not chosen to download pirated content. Conversely, every technical control could have succeeded and the same user behaviour would eventually find another vector. Security awareness training is not a soft control; it is an essential layer.

8.2.3 The 75-second problem

From the moment Rick double-clicked the dropper to the moment the ransom note appeared, only 75 seconds elapsed. No human can detect, triage, and respond to a ransomware infection in 75 seconds. Defence must be preventive, automated, and operating at machine speed.

8.2.4 Negative findings are findings

The absence of persistence is itself a forensic finding. The absence of cryptocurrency wallet addresses in memory is a forensic finding. Investigators must report what is *not* present with the same rigour as what is present, because absences shape attribution and response strategy.

8.3 Limitations

The investigation was limited by the data available:

- No disk image was provided, so file-system timeline analysis (\$MFT, \$LogFile) was not possible.
- No PCAP was provided, so the C2 communication content could not be inspected.
- No earlier memory snapshots were available, so the in-flight encryption process could not be observed.
- Reverse-engineering of the dropper binary at the assembly level was outside scope.

In a real-world incident, the investigation would be complemented by all of these data sources for a multi-perspective view.

8.4 Future Work

Suggested extensions of this investigation:

- Reverse-engineer the extracted dropper binary using Ghidra or IDA Pro to confirm the AES key derivation routine and potentially recover encrypted files.
- Develop a YARA rule based on the recovered ransom note string and the “confrim” typo as a unique fingerprint for this ransomware build.
- Build a Sigma rule based on the observed process tree (`explorer.exe` → user-named `.exe` from `\Torrents\` → child in `%TEMP%\RarSFX0\`) for SIEM detection of similar attacks.

- Extend the prevention model with measurable success criteria (e.g. “95% reduction in drop-per executions within 90 days of AppLocker deployment”).

8.5 Closing Statement

This investigation demonstrates that a structured, methodical application of memory forensics — supported by widely available open-source tools — can reconstruct even sophisticated ransomware attacks in complete detail. The same methodology used here against a CTF dataset is directly applicable to real-world incident response, with the only adjustments being scale, urgency, and the involvement of additional data sources. The skills practiced in this academic exercise translate directly to professional cyber forensics practice.

Chapter 9

Appendices

9.1 Appendix A — Complete Command Reference

This appendix lists every Volatility plugin invocation and shell command used during the investigation, organised by phase, for full reproducibility.

9.1.1 Phase 0 — Environment setup

```
mkdir -p ~/Cases/OtterCTF2018/{evidence,logs,output,reports,
    screenshots,working}
tree ~/Cases/OtterCTF2018/

# Hash original evidence
cd ~/Cases/OtterCTF2018/evidence
md5sum OtterCTF.vmem | tee OtterCTF.vmem.md5
sha256sum OtterCTF.vmem | tee OtterCTF.vmem.sha256

# Create working copy
cp OtterCTF.vmem ../working/OtterCTF_working.vmem

# Verify
sha256sum ~/Cases/OtterCTF2018/evidence/OtterCTF.vmem
sha256sum ~/Cases/OtterCTF2018/working/OtterCTF_working.vmem

# Read-only protect original
chmod 444 ~/Cases/OtterCTF2018/evidence/OtterCTF.vmem
```

9.1.2 Phase 1 — System profiling

```
cd ~/Cases/OtterCTF2018/working

# Vol2 imageinfo
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem imageinfo
```

```
# Vol3 confirmation
source ~/vol3-env/bin/activate
vol -f OtterCTF_working.vmem windows.info

# Hostname
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 printkey \
    -K "ControlSet001\Control\ComputerName\ComputerName"

# User context
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 envvars | grep -iE "username|userprofile" |
    grep -i rick

# Process tree
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 pstree

# Process list
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 pslist

# Network scan
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 netscan
```

9.1.3 Phase 2 — Attack vector identification

```
# Command-line evidence
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 cmdline

# File scan for Chrome history
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 filescan | grep -i "chrome.*history"

# Dump Chrome history
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 dumpfiles \
    -Q 0x000000007d45dcc0 \
    -D ../output/chrome_dump/ --name

# Query history (torrent)
sqlite3 ../output/chrome_dump/file.None.0xfffffa801a5193d0.History.
dat \
```

```
"SELECT url, title, visit_count, datetime(last_visit_time
    /1000000-11644473600,
'unixepoch') AS visit_time FROM urls WHERE url LIKE '%thepiratebay%'
    OR url
LIKE '%torrent%' ORDER BY last_visit_time DESC;"

# Dropper file scan and dump
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 filescan | grep -iE "rick.*morty|torrents"

python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 dumpfiles \
    -Q 0x7d63dbc0 \
    -D ../output/dropper/ --name

# Dropper hashing
sha256sum '../output/dropper/file.None.0xfffffa801a79c860.*'
md5sum '../output/dropper/file.None.0xfffffa801a79c860.*'
file '../output/dropper/file.None.0xfffffa801a79c860.*'
```

9.1.4 Phase 3 — Malware analysis

```
# Code injection scan
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 malfind --pid 3720

# DLL list
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 dlllist --pid 3720

# Hidden DLL detection
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 ldrmodules --pid 3720 | grep -E "False.*
False.*False"

# Process memory dump
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 memdump --pid 3720 -D ../output/
process_dumps/

# String analysis
strings -a -n 8 ../output/process_dumps/3720.dmp | grep -iE "bitcoin|
wallet"
strings -a -n 8 ../output/process_dumps/3720.dmp | grep -iE "encrypt|
password"
strings -a -n 8 -e 1 ../output/process_dumps/3720.dmp | grep -iE "
your files"
```

```
strings -a -n 8 -e 1 ../output/process_dumps/3720.dmp | grep -i "6
W66e0Cj"

# Credential dumping and cracking
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 hashdump

hashcat -m 1000 -a 0 /tmp/rick_hash.txt /usr/share/wordlists/rockyou.
txt --force -0
hashcat -m 1000 -a 0 /tmp/rick_hash.txt /tmp/rick_wordlist.txt --
force

# UserAssist
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 userassist

# Run keys
python2 ~/volatility2/vol.py -f OtterCTF_working.vmem \
    --profile=Win7SP1x64 printkey \
    -K "Software\Microsoft\Windows\CurrentVersion\Run"
```

9.2 Appendix B — Output File Inventory

The following output files were generated during the investigation and are preserved as case artifacts.

Table 9.1. Investigation output files

Filename	Description
01_imageinfo.txt	Volatility 2 image identification
01_vol3_info.txt	Volatility 3 confirmation
02_hostname.txt	Registry hostname extraction
02_envvars_filtered.txt	Filtered environment variables
03_pstree_vol2.txt	Process tree
03_pslist_vol2.txt	Process list
04_netscan.txt	Network connection scan
05_cmdline.txt	All process command lines
06_chrome_filescan.txt	Chrome history file objects
chrome_dump/	Extracted Chrome SQLite database (192 KB)
08_dropper_filescan.txt	Dropper file object location
dropper/	Extracted dropper binary (208 KB)
09_malfind_vmwaretray.txt	Code injection scan results
10_dlllist_vmwaretray.txt	Full DLL list of PID 3720
10_ldrmodules_vmwaretray.txt	Hidden DLL detection
process_dumps/3720.dmp	Full process memory dump (396 MB)
11_strings_*.txt	String analysis output (multiple)
12_encryption_key.txt	Recovered encryption key candidate
13_hashdump.txt	Extracted NTLM hashes
14_hashcat_*.txt	Hashcat attack logs
15_userassist.txt	UserAssist registry analysis
16_run_keys.txt	Run key enumeration

9.3 Appendix C — Indicators of Compromise

A consolidated IOC list for use in detection rules and threat intelligence sharing.

Table 9.2. Indicators of compromise

IOC type	Value
File name	Rick And Morty season 1 download.exe
File path	C:\Torrents\Rick And Morty season 1 download.exe
SHA-256	520161bec23819a713c2b467fe72f2378dc9a2808e176bc03f54cdbcf73f4b1d
MD5	f8a5d884dd0df20b7f4b20353fd6860d
Spawned process	vmware-tray.exe (PID 3720)
Spawned process path	%TEMP%\RarSFX0\vmware-tray.exe
Encrypted file extension	.WINDOWS
Ransom note filename	Flag.txt.WINDOWS
Ransom note marker string	The typo “confrim” (instead of “confirm”)
C2 endpoint	77.102.199.102:7575/tcp
C2 process	LunarMS.exe (PID 708)
Hidden DLL	mscorrc.dll loaded reflectively
Browsing precursor	Visits to thepiratebay3.org and lunarms.zapto.org

9.4 Appendix D — Glossary

CIRA

Critical Infrastructure Ransomware Attacks dataset — a curated catalogue of real-world ransomware incidents against critical infrastructure sectors.

EDR

Endpoint Detection and Response — a security technology category that monitors endpoint behaviour and responds to threats.

IOC

Indicator of Compromise — a forensic artifact (hash, IP, filename, registry key) that signals a security incident.

LSASS

Local Security Authority Subsystem Service — the Windows process that manages user authentication and stores credential hashes in memory.

MITRE ATT&CK

A globally recognised knowledge base of adversary tactics and techniques.

NTLM

NT LAN Manager — a Microsoft authentication protocol that produces password hashes recoverable from LSASS memory.

PEB

Process Environment Block — a Windows kernel structure containing per-process metadata including loaded modules.

Reflective DLL loading

An injection technique where a DLL is manually mapped into a process bypassing the standard `LoadLibrary` API.

RWX region

A memory region with simultaneous Read, Write, and Execute permissions — highly suspicious as legitimate code rarely needs this.

SFX

Self-extracting archive — an archive file that includes its own extraction stub, allowing it to extract without an external archiver.

Shellcode

Raw machine instructions injected into a process for execution, typically without an MZ header or PE structure.

Smash-and-grab

An attack pattern that prioritises speed and immediate impact over persistence or stealth.

UserAssist

A Windows registry mechanism that records every Explorer-launched executable, including

counts and focus times.

VAD

Virtual Address Descriptor — a kernel structure describing a region of process memory.

Volatility

An open-source memory forensics framework available in two major versions: 2 (legacy, Python 2) and 3 (modern, Python 3).

WDAC

Windows Defender Application Control — a Microsoft application allowlisting technology.

ZoneId

An NTFS alternate data stream tag indicating the security zone an executable was downloaded from. ZoneId=3 means “Internet”.

9.5 Appendix E — Acknowledgements

The team thanks the OtterCTF organisers for releasing this comprehensive dataset for educational use, the Volatility Foundation for maintaining the Volatility 2 and 3 frameworks, the MITRE Corporation for the ATT&CK framework, and the academic community for the body of memory forensics research that this investigation builds upon.